

```
In [2]: import pandas as pd
import seaborn as sns
```

```
In [3]: df=pd.read_csv("cleaned file.csv")
```

```
In [4]: df
```

Out[4]:

	Unnamed: 0	country_name	year	life_ladder	log_gdp_per_capita	social_support	health
0	0	Afghanistan	2008	3.724	7.370	0.451	
1	1	Afghanistan	2009	4.402	7.540	0.552	
2	2	Afghanistan	2010	4.758	7.647	0.539	
3	3	Afghanistan	2011	3.832	7.620	0.521	
4	4	Afghanistan	2012	3.783	7.705	0.521	
...	...	...	...	...	...	...	...
1920	1944	Zimbabwe	2016	3.735	7.984	0.768	
1921	1945	Zimbabwe	2017	3.638	8.016	0.754	
1922	1946	Zimbabwe	2018	3.616	8.049	0.775	
1923	1947	Zimbabwe	2019	2.694	7.950	0.759	
1924	1948	Zimbabwe	2020	3.160	7.829	0.717	

1925 rows × 12 columns

```
In [5]: df.describe()
```

Out[5]:

	Unnamed: 0	year	life_ladder	log_gdp_per_capita	social_support	health
count	1925.000000	1925.000000	1925.000000	1925.000000	1925.000000	
mean	973.437403	2013.231169	5.468297	9.366309	0.812817	
std	562.487417	4.162258	1.118544	1.143741	0.118451	
min	0.000000	2005.000000	2.375000	6.635000	0.290000	
25%	487.000000	2010.000000	4.640000	8.474000	0.750000	
50%	974.000000	2013.000000	5.387000	9.444000	0.836000	
75%	1462.000000	2017.000000	6.290000	10.335000	0.905000	
max	1948.000000	2020.000000	8.019000	11.648000	0.987000	

```
In [6]: df.median(numeric_only=True)
```

```
Out[6]: Unnamed: 0          974.000
        year          2013.000
        life_ladder          5.387
        log_gdp_per_capita      9.444
        social_support          0.836
        healthy_life_expectancy_at_birth 65.000
        freedom_to_make_life_choices      0.760
        generosity          -0.016
        perceptions_of_corruption      0.794
        positive_affect          0.722
        negative_affect          0.258
        dtype: float64
```

```
In [7]: df.mean(numeric_only=True)
```

```
Out[7]: Unnamed: 0          973.437403
        year          2013.231169
        life_ladder          5.468297
        log_gdp_per_capita      9.366309
        social_support          0.812817
        healthy_life_expectancy_at_birth 63.353340
        freedom_to_make_life_choices      0.742469
        generosity          0.000619
        perceptions_of_corruption      0.746940
        positive_affect          0.710019
        negative_affect          0.268366
        dtype: float64
```

```
In [8]: df["life_ladder"].quantile([0.25,0.5,0.75])
```

```
Out[8]: 0.25    4.640
        0.50    5.387
        0.75    6.290
        Name: life_ladder, dtype: float64
```

```
In [9]: df.quantile([0.25,0.5,0.75],numeric_only=True)
```

```
Out[9]:
```

	Unnamed: 0	year	life_ladder	log_gdp_per_capita	social_support	healthy_life_exp
0.25	487.0	2010.0	4.640	8.474	0.750	
0.50	974.0	2013.0	5.387	9.444	0.836	
0.75	1462.0	2017.0	6.290	10.335	0.905	

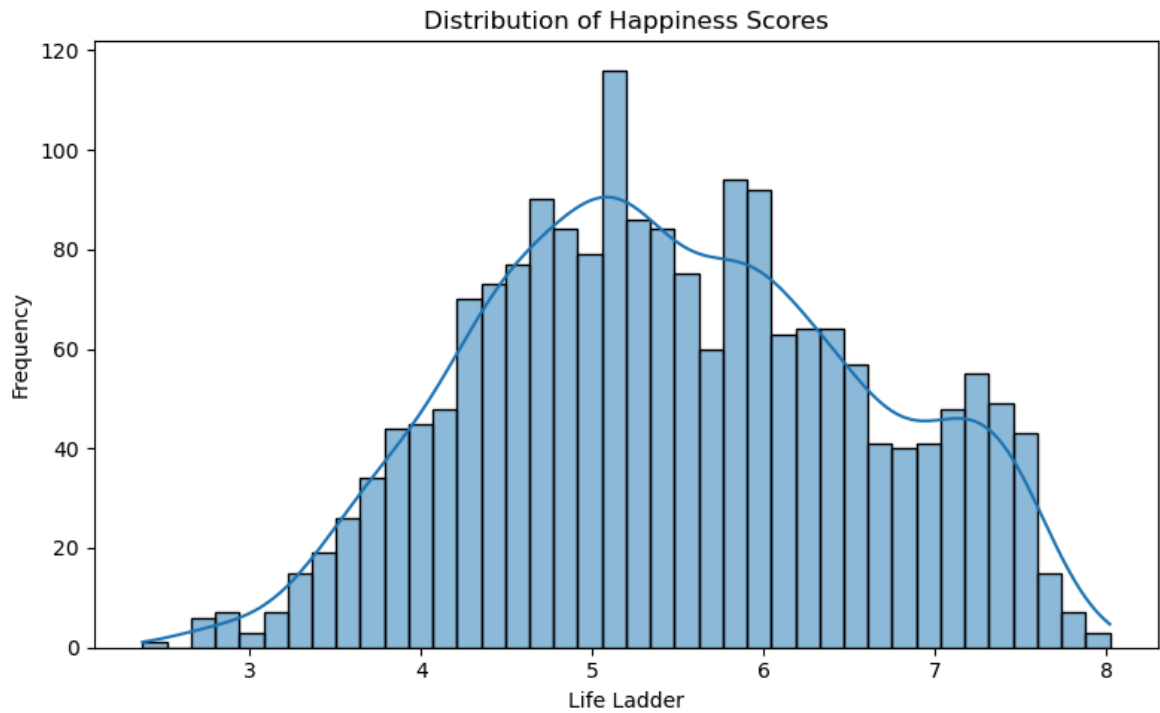
```
In [10]: df = df.drop("Unnamed: 0", axis=1)
```

Normal Distribution

```
In [11]: df.value_counts()
```

```
Out[11]: country_name year life_ladder log_gdp_per_capita social_support healthy_li
fe_expectancy_at_birth freedom_to_make_life_choices generosity perceptions_o
f_corruption positive_affect negative_affect
Afghanistan 2008 3.724 7.370 0.451 50.80
0.718 0.168 0.882 0.518
0.258 1
Netherlands 2014 7.321 10.867 0.909 71.88
0.910 0.331 0.457 0.868
0.221 1
Norway 2017 7.579 11.050 0.950 73.10
0.953 0.236 0.250 0.849
0.203 1
2016 7.596 11.035 0.960 73.00
0.954 0.133 0.410 0.850
0.209 1
2015 7.603 11.033 0.947 72.90
0.948 0.257 0.299 0.843
0.209 1
..
Greece 2014 4.756 10.257 0.832 71.74
0.369 -0.288 0.930 0.695
0.385 1
2013 4.720 10.243 0.687 71.68
0.426 -0.272 0.941 0.689
0.482 1
2012 5.096 10.268 0.812 71.62
0.373 -0.305 0.959 0.581
0.352 1
2011 5.372 10.339 0.852 71.56
0.528 -0.316 0.941 0.591
0.323 1
Zimbabwe 2020 3.160 7.829 0.717 56.80
0.643 -0.009 0.789 0.703
0.346 1
Name: count, Length: 1925, dtype: int64
```

```
In [12]: import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))
sns.histplot(df["life_ladder"], bins=40, kde=True)
plt.title("Distribution of Happiness Scores")
plt.xlabel("Life Ladder")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```



```
In [13]: import matplotlib.pyplot as plt

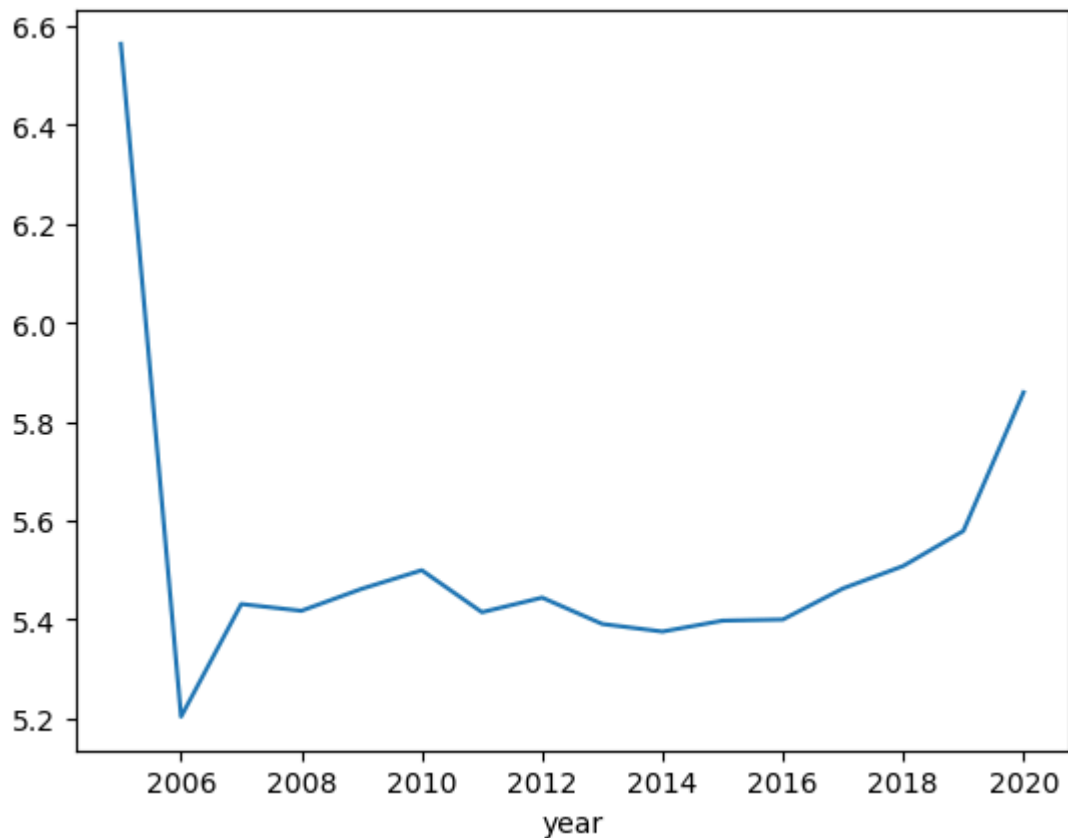
plt.hist(df["life_ladder"], bins=82)
plt.xlabel("life_ladder")
plt.ylabel("Frequency")
plt.title("Distribution of Happiness")
plt.show()
```



HAPPINES OVER YEARS

```
In [14]: df.groupby("year")["life_ladder"].mean().plot()
```

```
Out[14]: <Axes: xlabel='year'>
```



Happiness levels decreased sharply in the early years, remained relatively stable for a decade, and showed gradual improvement in recent years.

```
In [15]: df["life_ladder"].skew()
```

```
Out[15]: np.float64(0.06466158095247462)
```

```
In [16]: df["healthy_life_expectancy_at_birth"].skew()
```

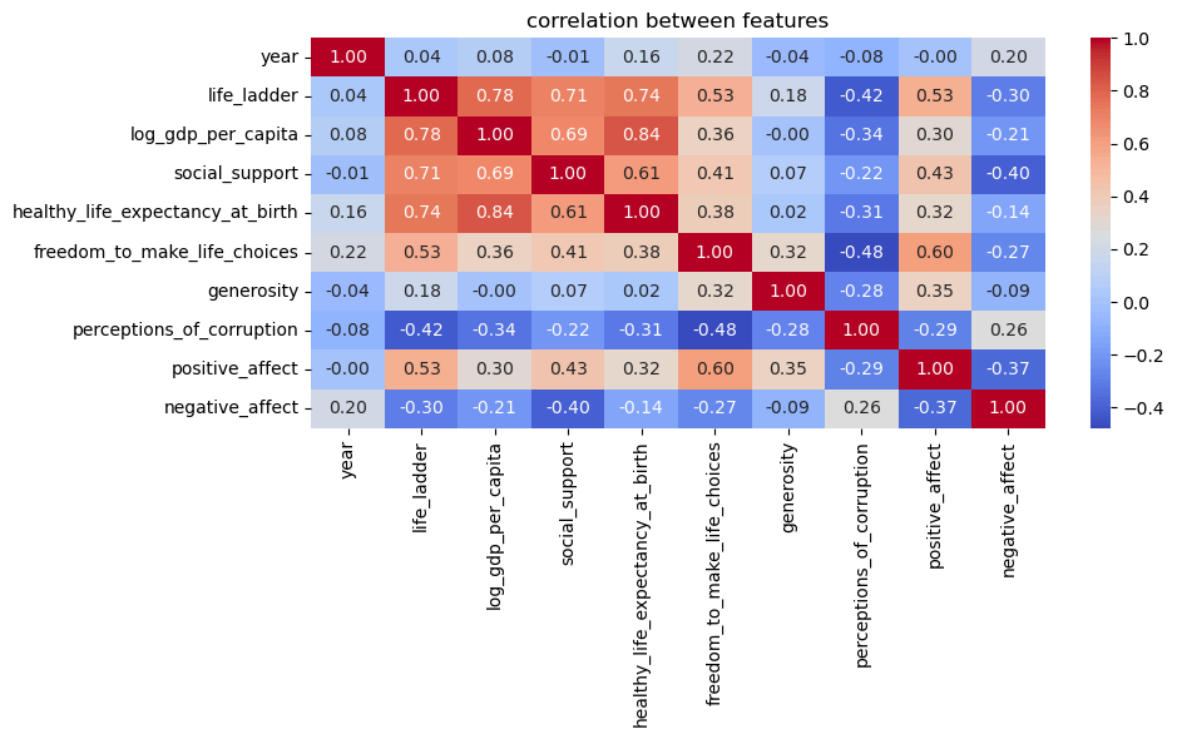
```
Out[16]: np.float64(-0.7514343287731668)
```

Correlation

```
In [17]: plt.figure(figsize=(10,6))

sns.heatmap(df.corr(numeric_only=True), annot=True, fmt=".2f", cmap="coolwarm")

plt.title("correlation between features")
plt.tight_layout()
plt.show()
\
```



Insights

- Strong Positive Relationships (+): life_ladder ↔ log_gdp_per_capita : ~0.78, life_ladder ↔ social_support : ~0.71, life_ladder ↔ healthy_life_expectancy_at_birth : ~0.74, log_gdp_per_capita ↔ healthy_life_expectancy_at_birth : ~0.84, social_support ↔ log_gdp_per_capita : ~0.69, freedom_to_make_life_choices ↔ positive_affect : ~0.60

- Strong Negative Relationships (-): life_ladder ↔ perceptions_of_corruption : ~ -0.42, freedom_to_make_life_choices ↔ perceptions_of_corruption : ~ -0.48, social_support ↔ negative_affect : ~ -0.40, positive_affect ↔ negative_affect : ~ -0.37

- Moderate Relationships: life_ladder ↔ freedom_to_make_life_choices : ~0.53, life_ladder ↔ positive_affect : ~0.53, log_gdp_per_capita ↔ social_support : ~0.69, healthy_life_expectancy_at_birth ↔ social_support : ~0.61, generosity ↔ positive_affect : ~0.35

- Weak / No Relationships: year ↔ most variables : very weak correlation, generosity ↔ most variables : weak correlation, log_gdp_per_capita ↔ generosity : ~0.00 (no relation)

Globally, happiness (life_ladder) is strongly positively related to GDP, social support, and life expectancy, while it is negatively affected by corruption and negative emotions.

Pie Chart

```
In [18]: import matplotlib.pyplot as plt
import pandas as pd

df["happiness_level"] = pd.cut(
```

```

df["life_ladder"],
bins=[0, 4, 6, 10],
labels=["Low", "avarage", "High"]
)

counts = df["happiness_level"].value_counts()

plt.figure(figsize=(7,7))

colors = ["#ff9999", "#66b3ff", "#99ff99"]
explode = (0.05, 0.05, 0.08)

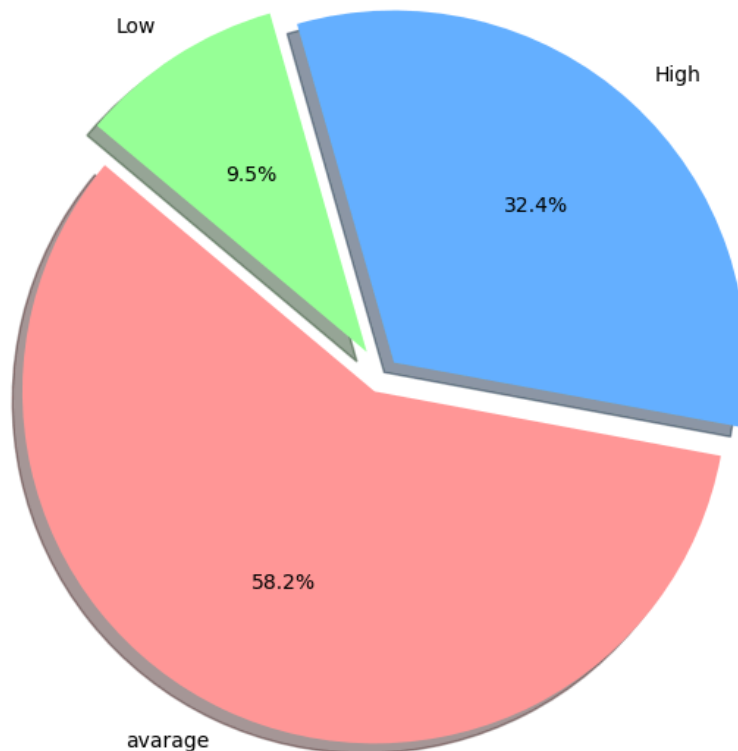
plt.pie(
    counts,
    labels=counts.index,
    autopct="%1.1f%%",
    startangle=500,
    colors=colors,
    explode=explode,
    shadow=True
)

plt.title("Distribution of Happiness Levels", fontsize=35)
plt.axis("equal")

plt.show()

```

Distribution of Happiness Levels

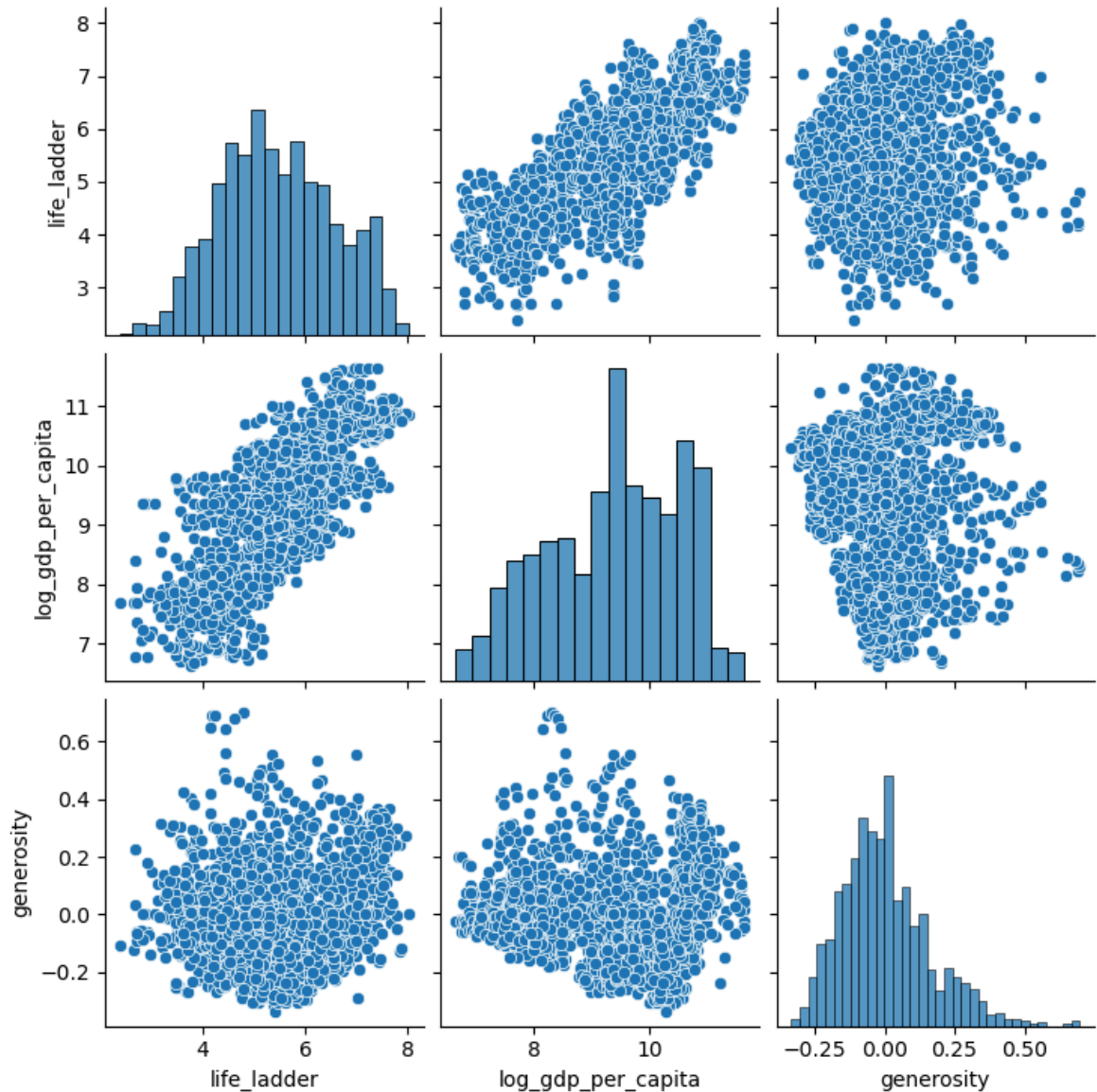


Majority of countries fall in the Medium happiness category. A significant portion has High happiness levels. Very few countries are in the Low happiness category. Happiness distribution is skewed towards moderate levels

RELATIONS

```
In [19]: import seaborn as sns
sns.pairplot(df[["life_ladder", "log_gdp_per_capita", "generosity"]])
```

Out[19]: <seaborn.axisgrid.PairGrid at 0x27f3f929940>



```
In [20]: import seaborn as sns
import matplotlib.pyplot as plt

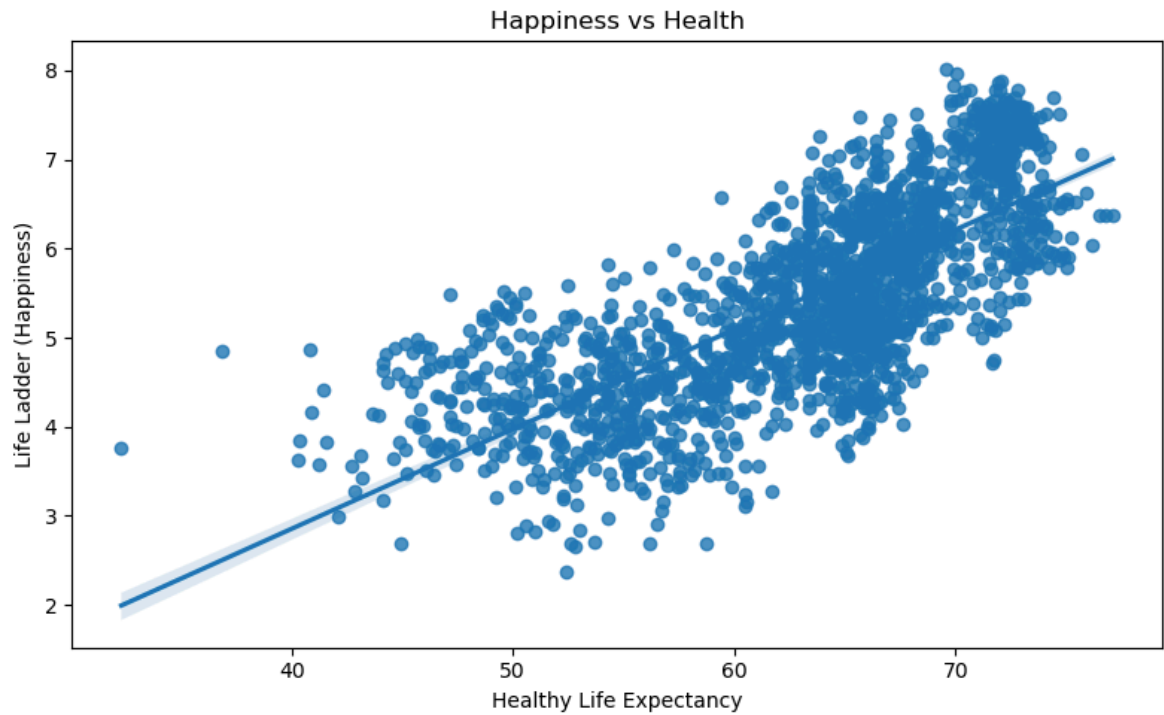
plt.figure(figsize=(8,5))

sns.regplot(
    data=df,
    x="healthy_life_expectancy_at_birth",
    y="life_ladder"
)

plt.title("Happiness vs Health")
plt.xlabel("Healthy Life Expectancy")
plt.ylabel("Life Ladder (Happiness)")
```

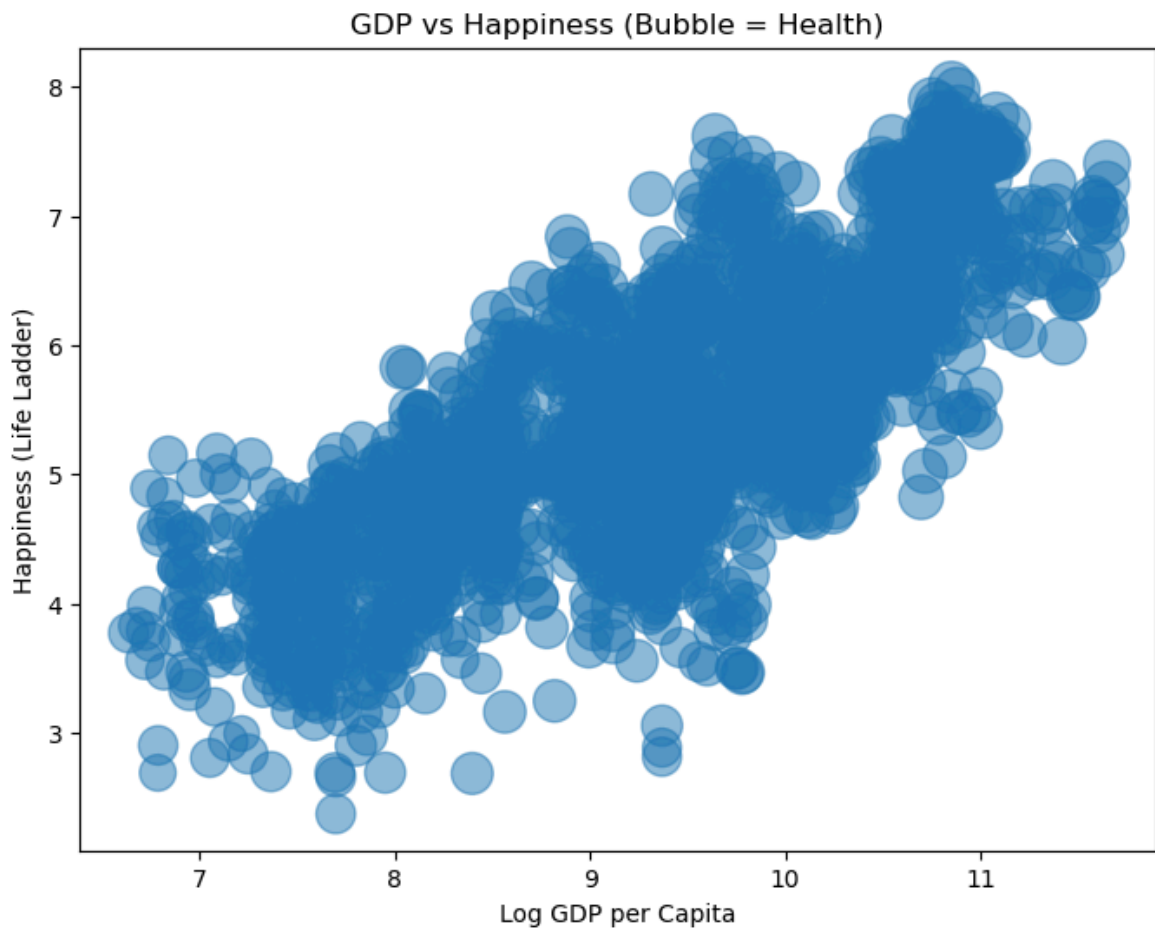


```
plt.tight_layout()
plt.show()
```

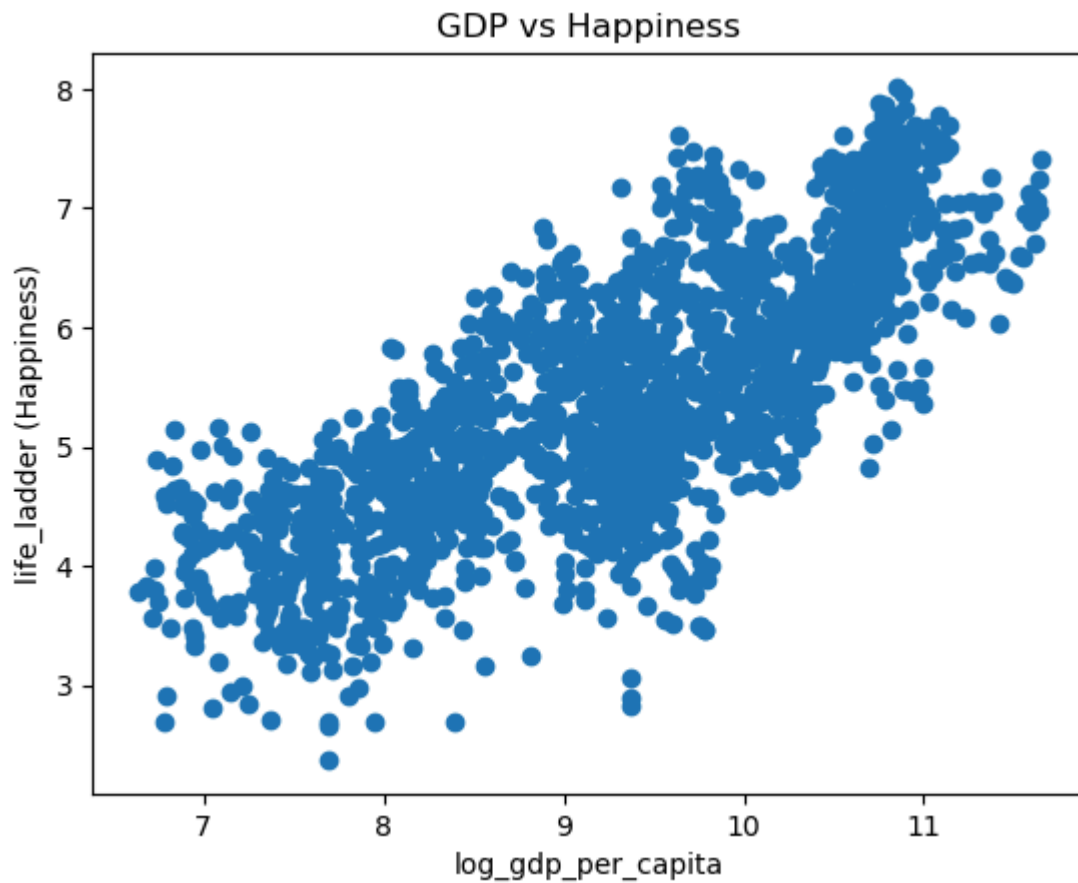


happines and health is directly propotional

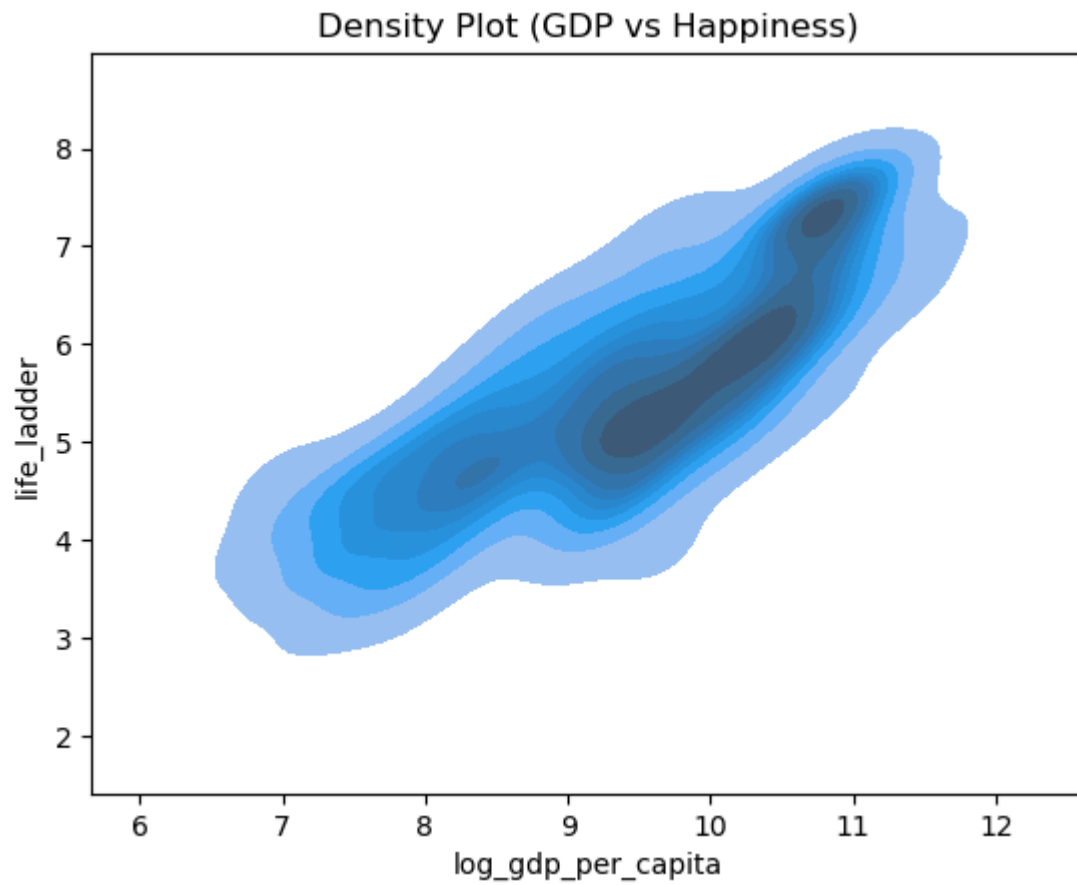
```
In [21]: import matplotlib.pyplot as plt
plt.figure(figsize=(8,6))
plt.scatter(
    df["log_gdp_per_capita"],
    df["life_ladder"],
    s=df["healthy_life_expectancy_at_birth"] * 5,
    alpha=0.5
)
plt.xlabel("Log GDP per Capita")
plt.ylabel("Happiness (Life Ladder)")
plt.title("GDP vs Happiness (Bubble = Health)")
plt.show()
```



```
In [22]: import matplotlib.pyplot as plt
plt.scatter(df["log_gdp_per_capita"], df["life_ladder"])
plt.xlabel("log_gdp_per_capita")
plt.ylabel("life_ladder (Happiness)")
plt.title("GDP vs Happiness")
plt.show()
```

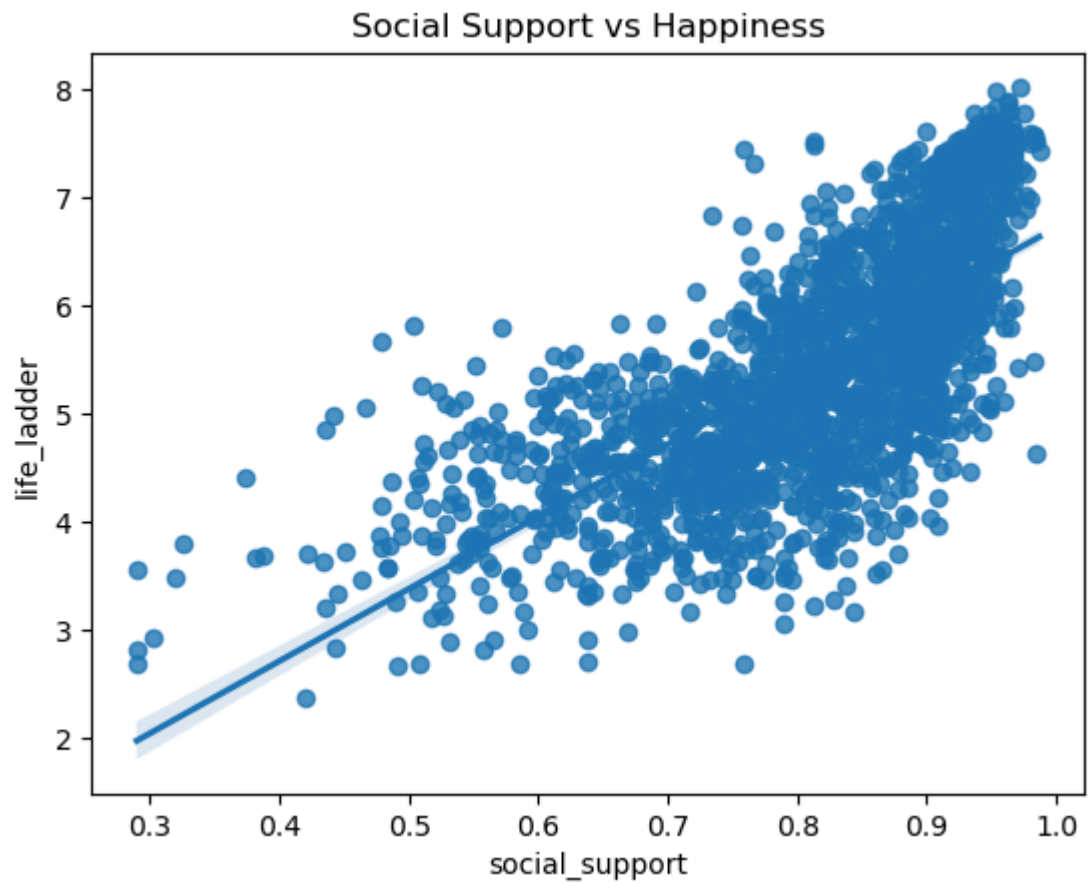


```
In [23]: sns.kdeplot(  
    data=df,  
    x="log_gdp_per_capita",  
    y="life_ladder",  
    fill=True  
)  
  
plt.title("Density Plot (GDP vs Happiness)")  
plt.show()
```



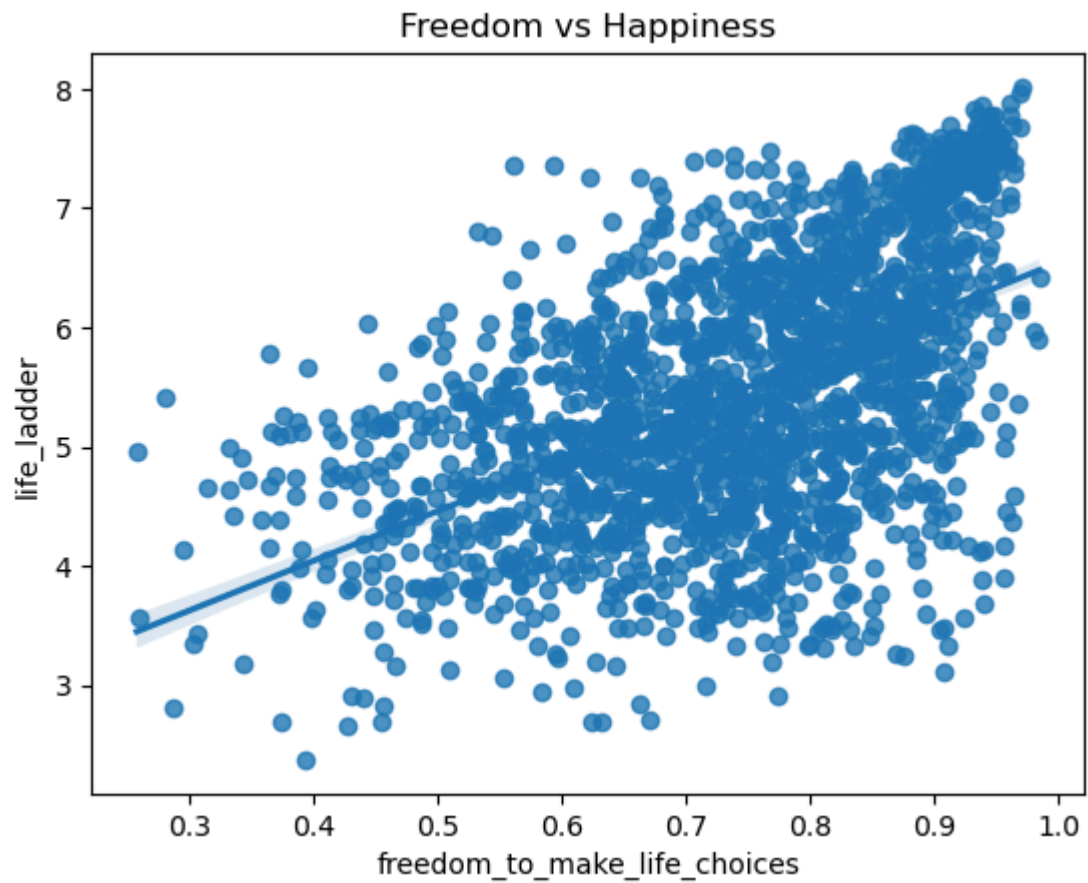
gdp and happiness are directly propotional

```
In [24]: sns.regplot(data=df, x="social_support", y="life_ladder")  
plt.title("Social Support vs Happiness")  
plt.show()
```



Social support and Happiness directly propotional

```
In [25]: sns.regplot(data=df, x="freedom_to_make_life_choices", y="life_ladder")  
plt.title("Freedom vs Happiness")  
plt.show()
```



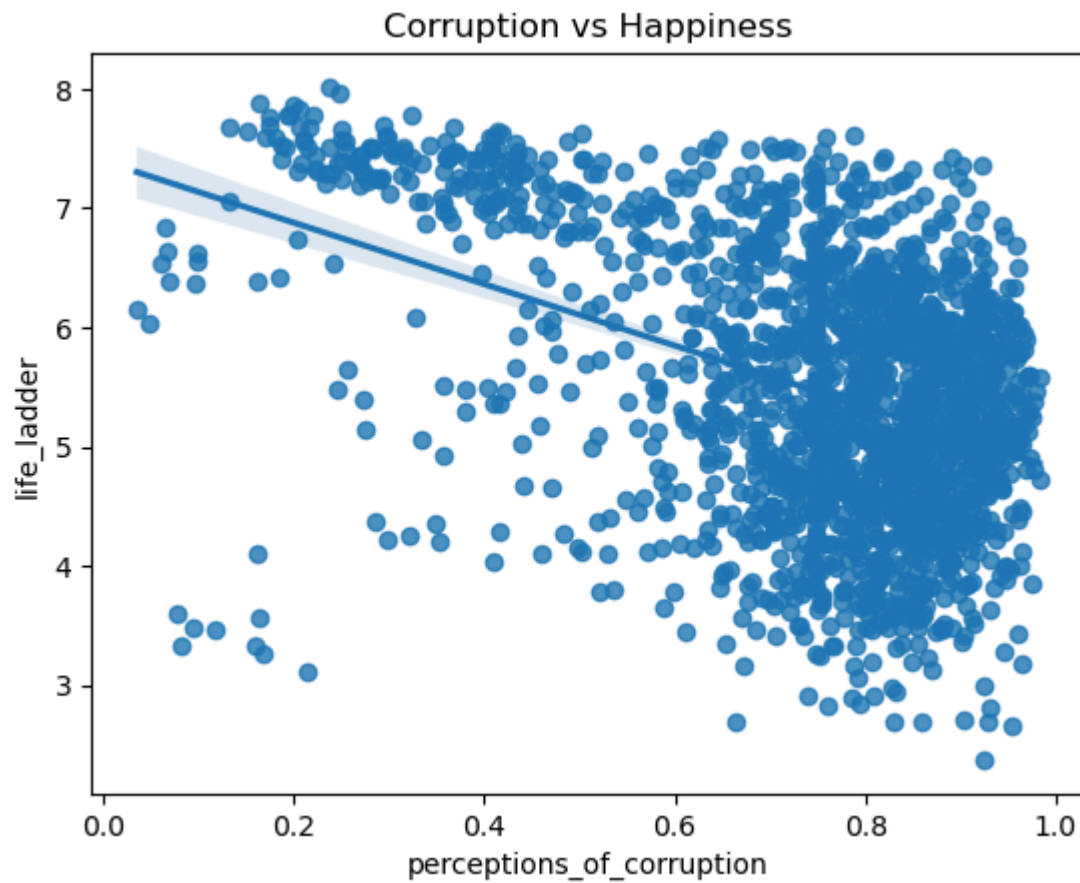
Freedom and Happiness is directly propotional

```
In [26]: sns.scatterplot(data=df, x="generosity", y="life_ladder")  
plt.title("Generosity vs Happiness")  
plt.show()
```



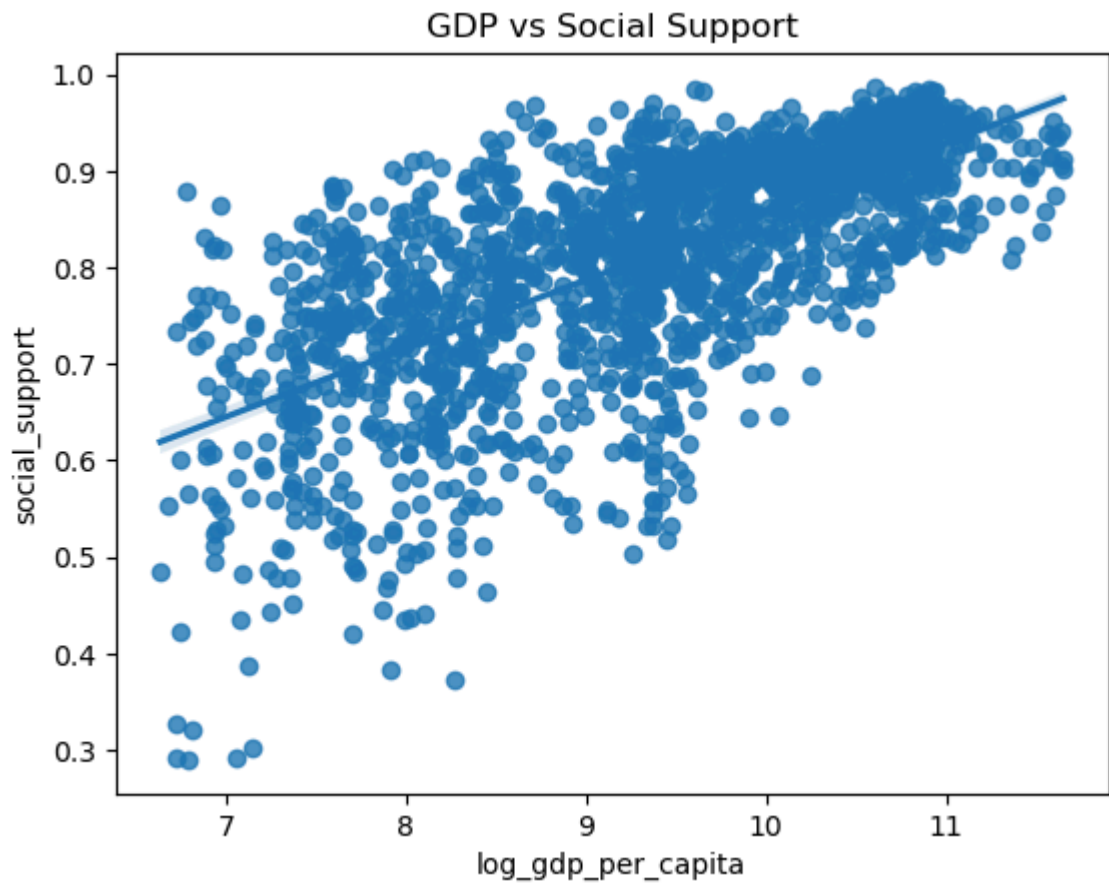
There is no big propotianility between generosity and Happiness

```
In [27]: sns.regplot(data=df, x="perceptions_of_corruption", y="life_ladder")  
plt.title("Corruption vs Happiness")  
plt.show()
```



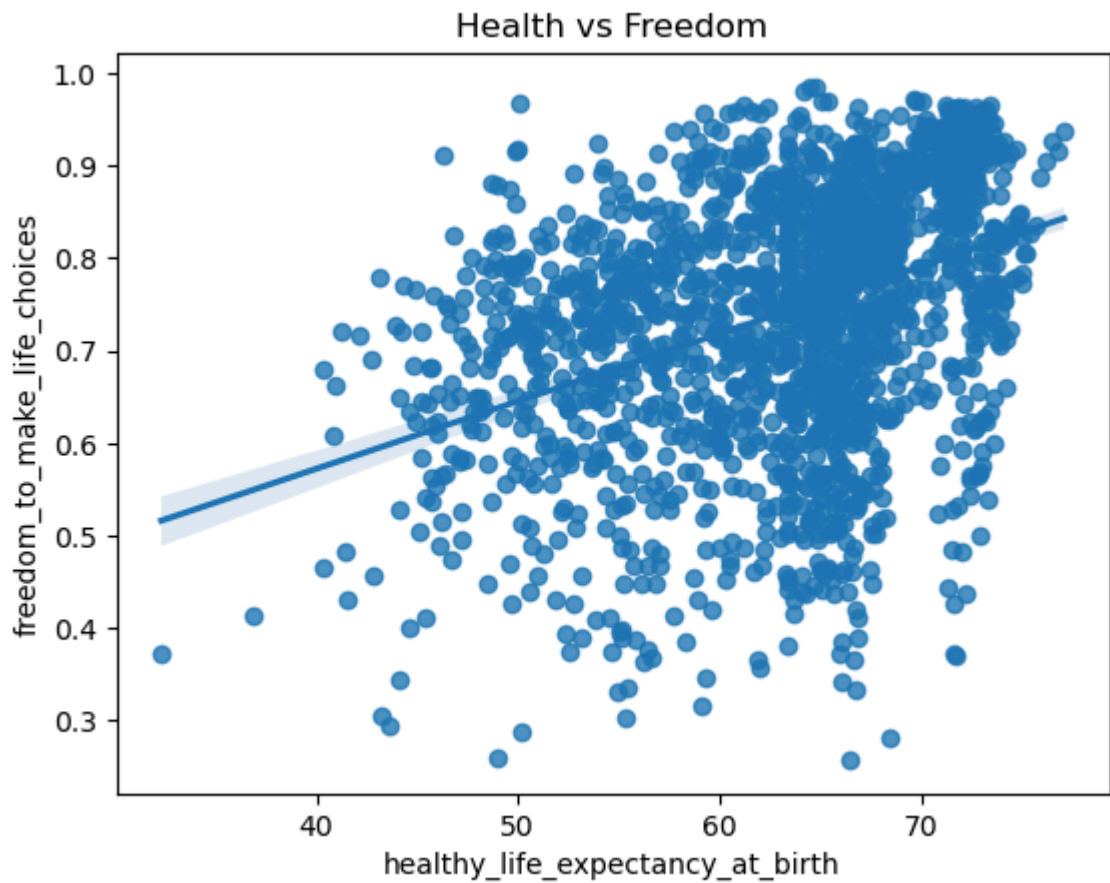
here happiness is inversely propotional to corruption

```
In [28]: sns.regplot(data=df, x="log_gdp_per_capita", y="social_support")  
plt.title("GDP vs Social Support")  
plt.show()
```

gdp and social support is directly propotional

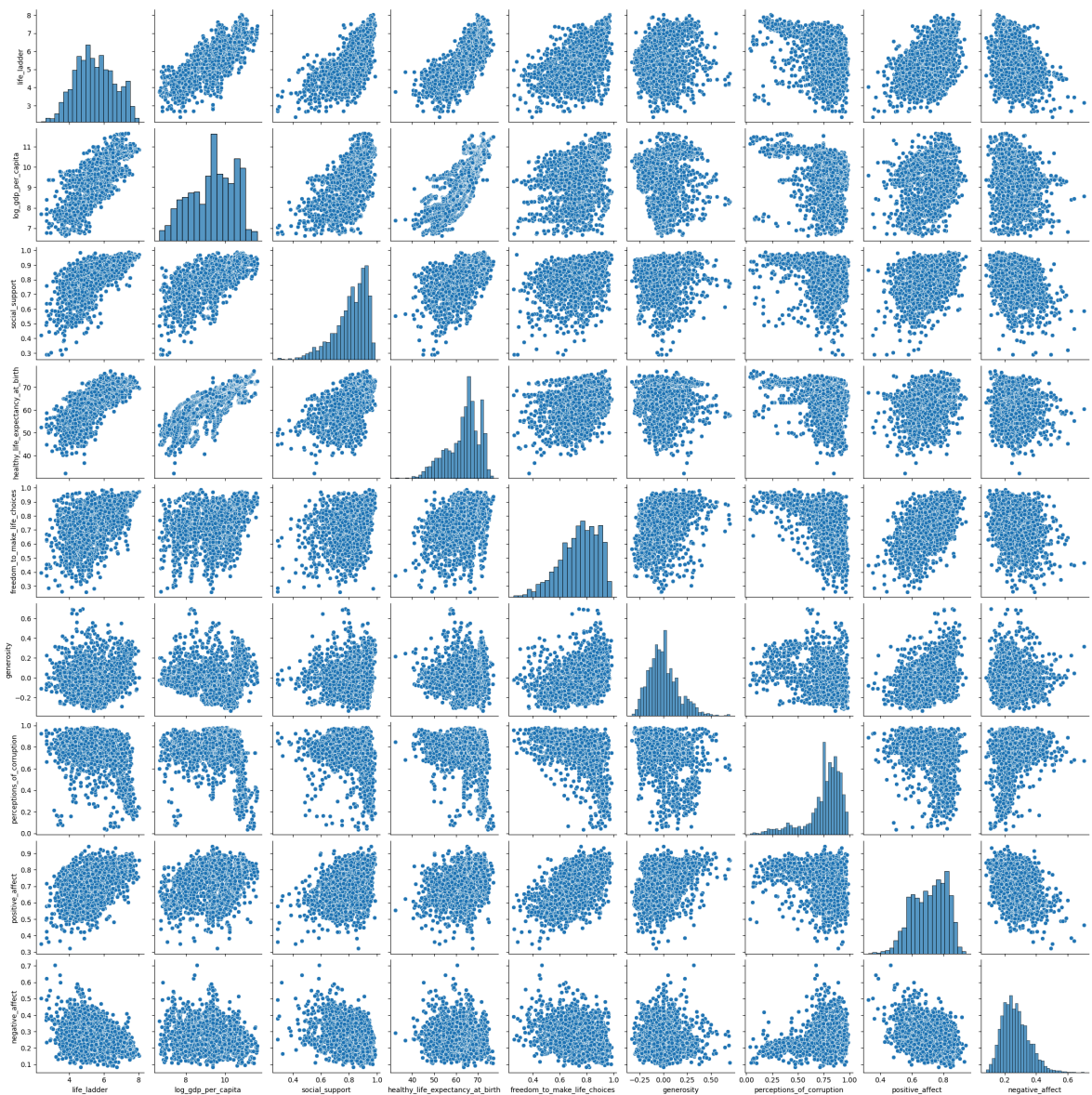
```
In [29]: sns.regplot(data=df, x="healthy_life_expectancy_at_birth", y="freedom_to_make_li  
plt.title("Health vs Freedom")  
plt.show()
```



health and freedom is directly propotional

```
In [30]: sns.pairplot(df[[  
    "life_ladder",  
    "log_gdp_per_capita",  
    "social_support",  
    "healthy_life_expectancy_at_birth",  
    "freedom_to_make_life_choices",  
    "generosity", "perceptions_of_corruption", "positive_affect", "negative_affect"  
]])
```

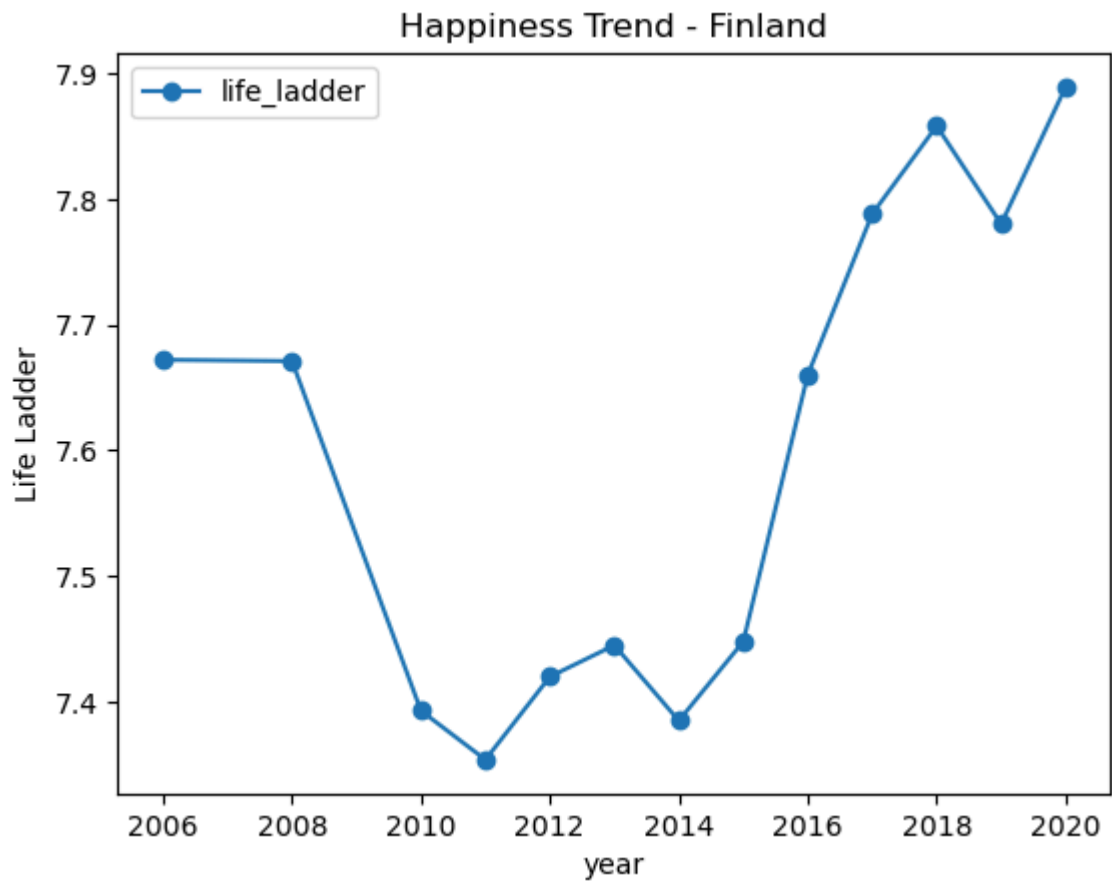
```
Out[30]: <seaborn.axisgrid.PairGrid at 0x27f402eac10>
```



ALL IN ONE

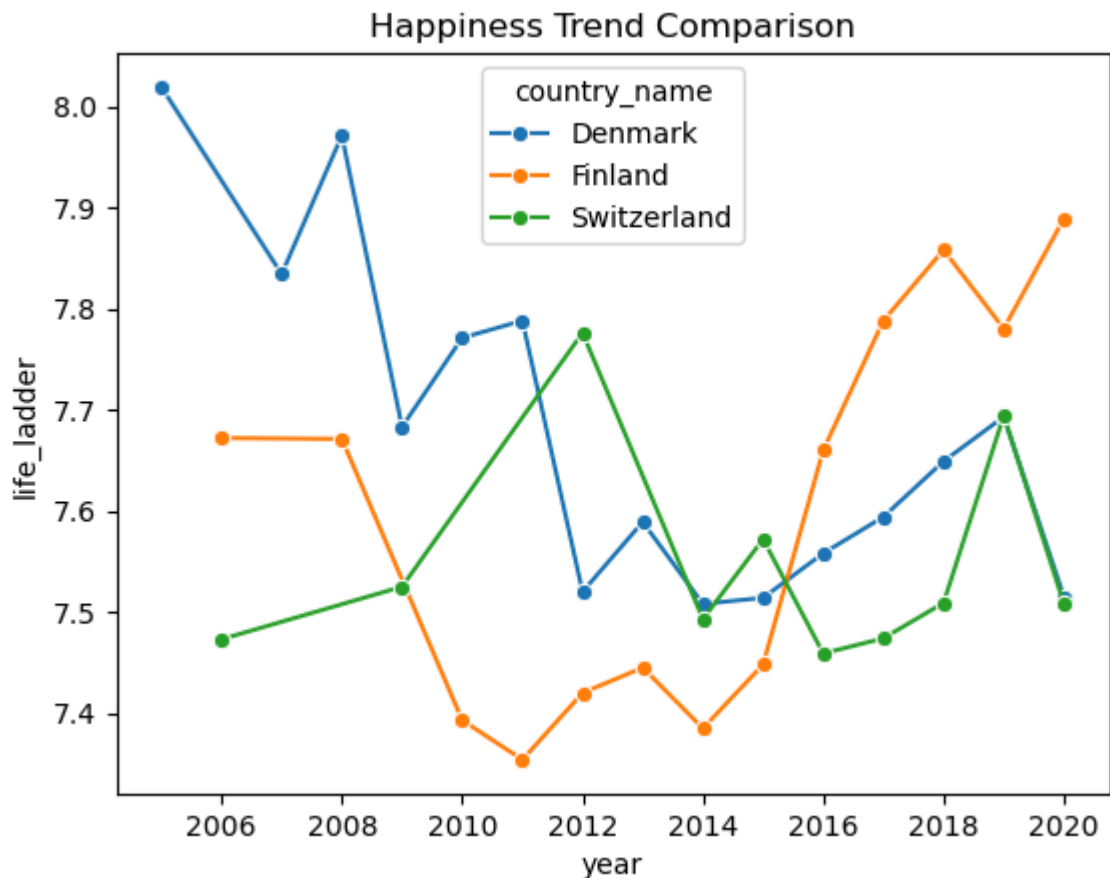
Country-wise Analysis

```
In [31]: country_data = df[df["country_name"] == "Finland"]
country_data.sort_values("year").plot(
    x="year", y="life_ladder", kind="line", marker="o"
)
import matplotlib.pyplot as plt
plt.title("Happiness Trend - Finland")
plt.ylabel("Life Ladder")
plt.show()
```



the happiness trend in finland over years is increasing

```
In [32]: countries = ["Finland", "Denmark", "Switzerland"]
subset = df[df["country_name"].isin(countries)]
import seaborn as sns
sns.lineplot(
    data=subset,
    x="year",
    y="life_ladder",
    hue="country_name",
    marker="o"
)
plt.title("Happiness Trend Comparison")
plt.show()
```



the happiness trend in some countries over years. the happiness trend Denmark decreasing over years. the happiness trend in Finland increasing over years. the happiness trend in decreasing and increasing over years

```
In [33]: afg_df = df[df["country_name"] == "Afghanistan"]
```

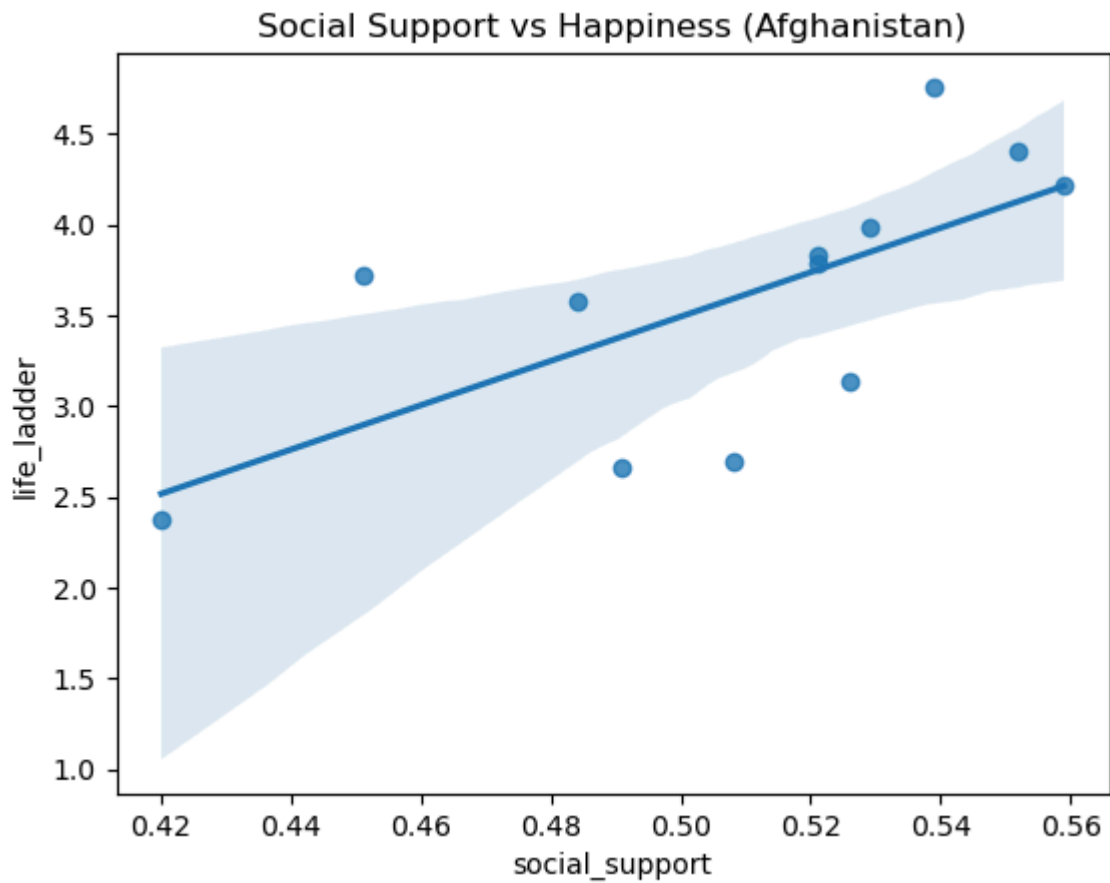
```
In [34]: afg_df.head()
```

```
Out[34]:
```

	country_name	year	life_ladder	log_gdp_per_capita	social_support	healthy_life_exp
0	Afghanistan	2008	3.724	7.370	0.451	
1	Afghanistan	2009	4.402	7.540	0.552	
2	Afghanistan	2010	4.758	7.647	0.539	
3	Afghanistan	2011	3.832	7.620	0.521	
4	Afghanistan	2012	3.783	7.705	0.521	

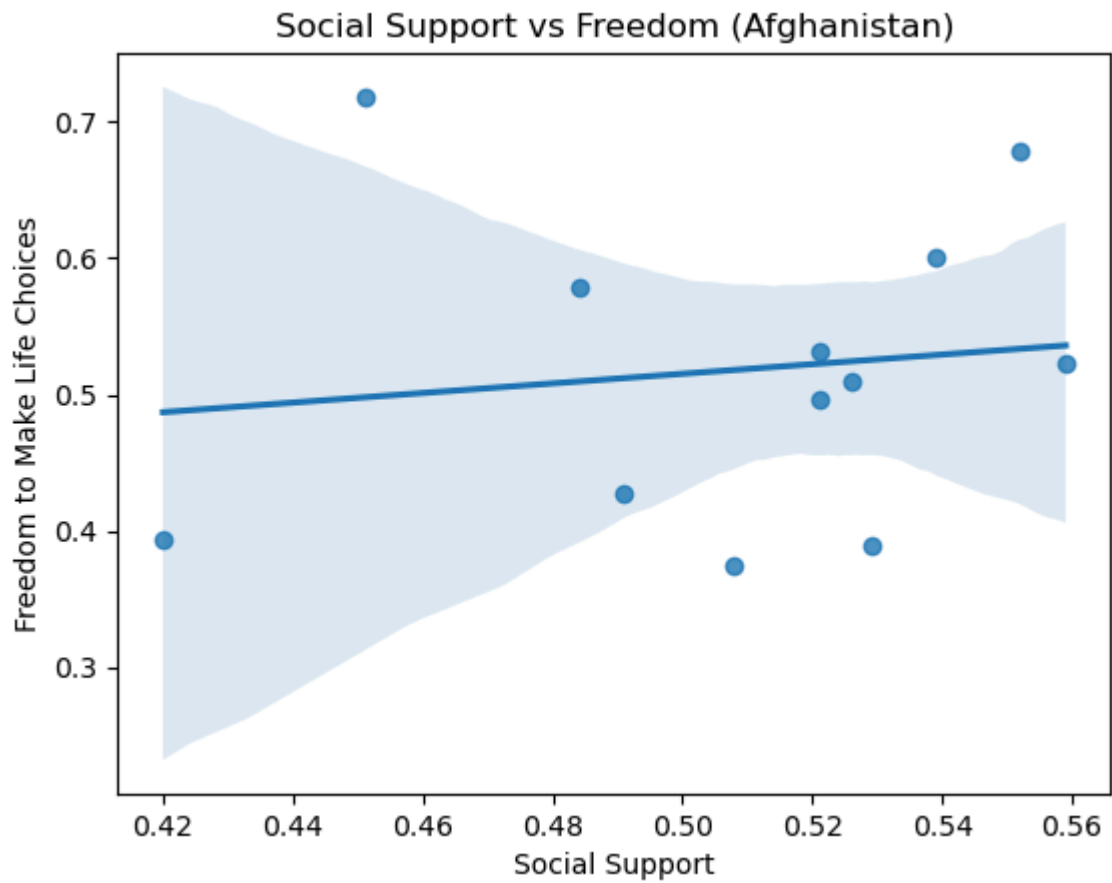
```
In [35]: sns.regplot(
data=afg_df,
x="social_support",
y="life_ladder"
)

plt.title("Social Support vs Happiness (Afghanistan)")
plt.show()
```



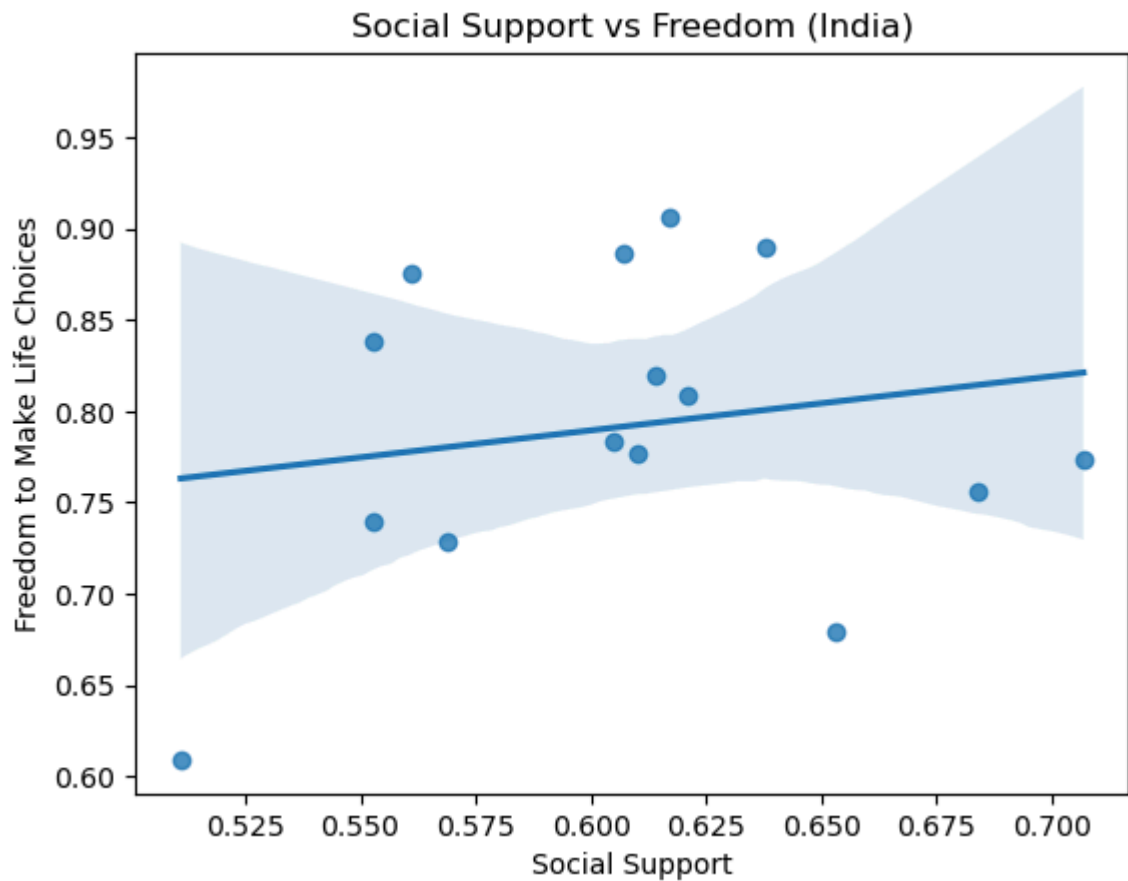
the relation btw happiness and social support in Afghanistan is directly propotional

```
In [36]: sns.regplot(  
    data=afg_df,  
    x="social_support",  
    y="freedom_to_make_life_choices"  
)  
plt.title("Social Support vs Freedom (Afghanistan)")  
plt.xlabel("Social Support")  
plt.ylabel("Freedom to Make Life Choices")  
plt.show()
```



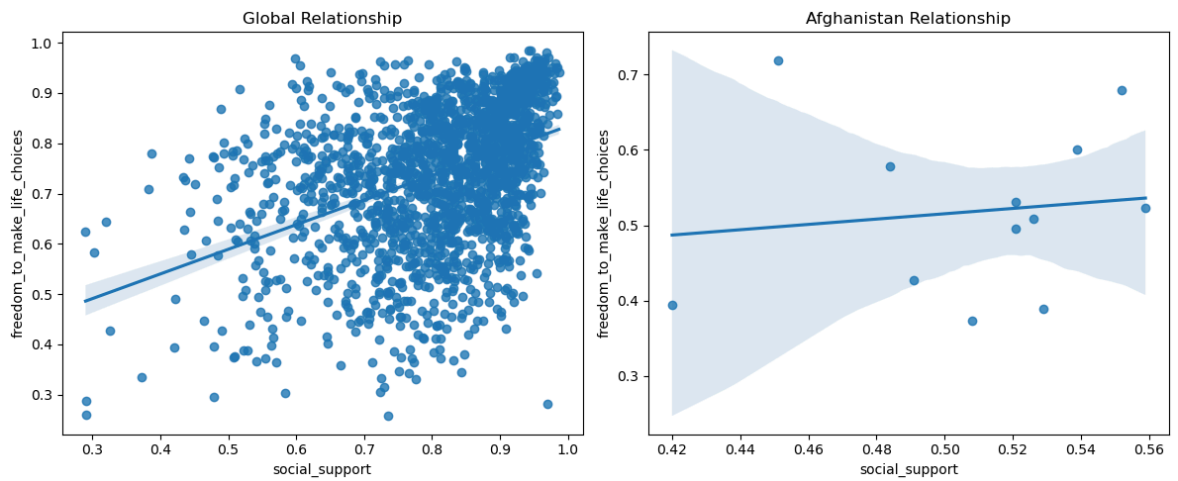
```
In [37]: ind_df = df[df["country_name"] == "India"]
```

```
In [38]: sns.regplot(  
    data=ind_df,  
    x="social_support",  
    y="freedom_to_make_life_choices"  
)  
plt.title("Social Support vs Freedom (India)")  
plt.xlabel("Social Support")  
plt.ylabel("Freedom to Make Life Choices")  
plt.show()
```



the relation btw Freedom and social support in India

```
In [39]: import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
sns.regplot(
data=df,
x="social_support",
y="freedom_to_make_life_choices"
)
plt.title("Global Relationship")
plt.subplot(1,2,2)
sns.regplot(
data=afg_df,
x="social_support",
y="freedom_to_make_life_choices"
)
plt.title("Afghanistan Relationship")
plt.tight_layout()
plt.show()
```

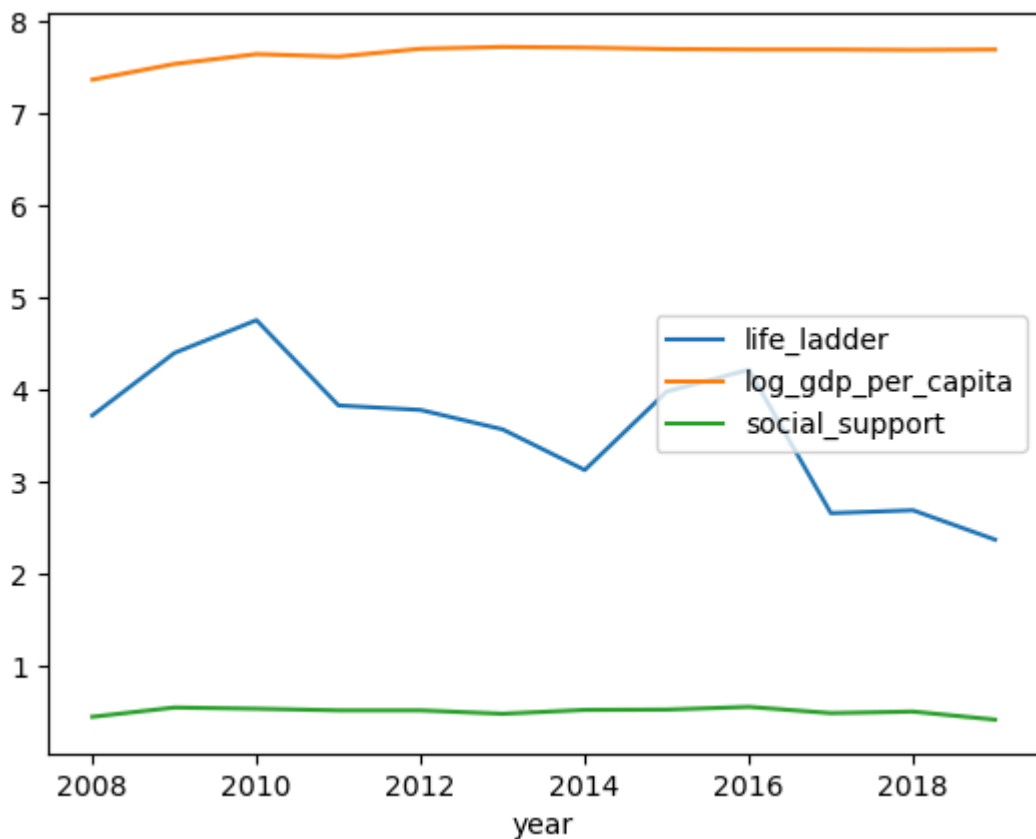



comparing the relation graph between Globally and Afghanistan

In []:

```
In [40]: afg_df.sort_values("year").plot(
x="year",
y=["life_ladder", "log_gdp_per_capita", "social_support"])
```

Out[40]: <Axes: xlabel='year'>



Happiness in Afghanistan fluctuates over time, while GDP and social support show gradual trends, indicating that happiness is influenced by multiple factors beyond economic growth

In [41]: df

Out[41]:

	country_name	year	life_ladder	log_gdp_per_capita	social_support	healthy_life_e
0	Afghanistan	2008	3.724	7.370	0.451	
1	Afghanistan	2009	4.402	7.540	0.552	
2	Afghanistan	2010	4.758	7.647	0.539	
3	Afghanistan	2011	3.832	7.620	0.521	
4	Afghanistan	2012	3.783	7.705	0.521	
...	...	...	...	...	...	...
1920	Zimbabwe	2016	3.735	7.984	0.768	
1921	Zimbabwe	2017	3.638	8.016	0.754	
1922	Zimbabwe	2018	3.616	8.049	0.775	
1923	Zimbabwe	2019	2.694	7.950	0.759	
1924	Zimbabwe	2020	3.160	7.829	0.717	

1925 rows × 12 columns



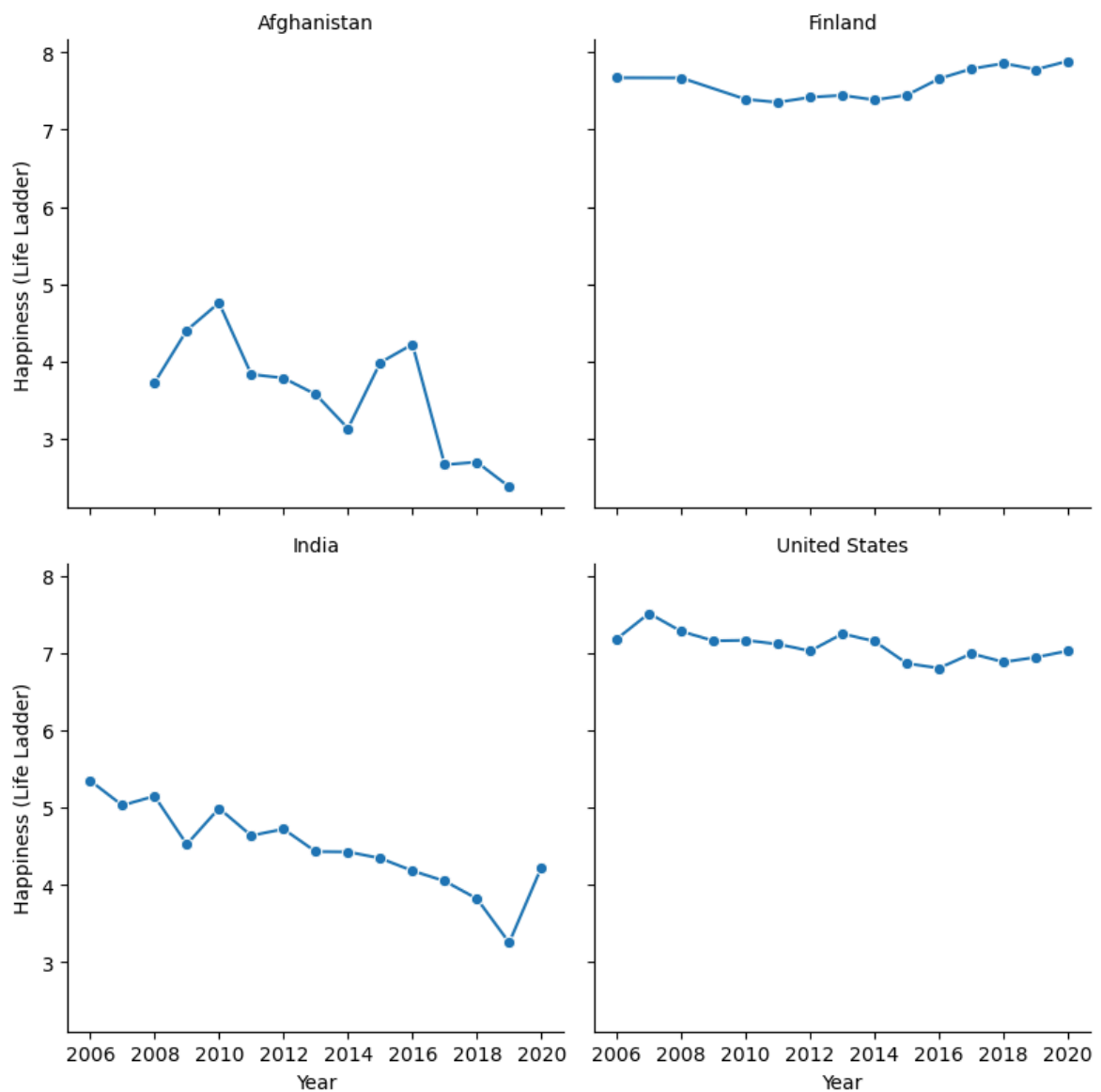
In [42]:

```
afg_df["life_ladder"].std()
df["life_ladder"].std()
```

Out[42]: 1.1185439019174808

In [43]:

```
countries = ["Afghanistan", "India", "Finland", "United States"]
subset = df[df["country_name"].isin(countries)]
g = sns.FacetGrid(subset, col="country_name", col_wrap=2, height=4)
g.map(sns.lineplot, "year", "life_ladder", marker="o")
g.set_titles("{col_name}")
g.set_axis_labels("Year", "Happiness (Life Ladder)")
plt.tight_layout()
plt.show()
```



Finland leads in happiness with stable trends, the United States remains high with slight variation, India shows gradual improvement, and Afghanistan has the lowest and most unstable happiness levels. Developed countries tend to have higher and more stable happiness levels, while developing or conflict-affected countries show lower and more fluctuating trends.

```
In [44]: top10 = (
df.groupby("country_name")["life_ladder"]
.mean()
.sort_values(ascending=False)
.head(10)
)
print(top10)
```

```
country_name
Denmark      7.680400
Finland      7.597154
Switzerland  7.548300
Norway       7.512400
Netherlands  7.466286
Iceland      7.446500
Canada       7.376333
Sweden       7.369467
New Zealand  7.310286
Australia    7.282071
Name: life_ladder, dtype: float64
```

```
In [45]: country_mean = df.groupby("country_name")["life_ladder"].mean()
rank = country_mean.rank(ascending=False)
india_rank = rank["India"]
print("India's Rank:", int(india_rank))
```

India's Rank: 126

```
In [46]: country_mean = df.groupby("country_name")["life_ladder"].mean()
last_country = country_mean.idxmin()
print("Last Country:", last_country)
```

Last Country: South Sudan

```
In [47]: df_2020 = df[df['year'] == 2020]
df_2020 = df_2020.sort_values(by='life_ladder', ascending=False)
df_2020['rank'] = range(1, len(df_2020) + 1)
argentina_rank = df_2020[df_2020['country_name'] == 'Argentina']
print(argentina_rank[['country_name', 'life_ladder', 'rank']])
```

```
country_name  life_ladder  rank
50    Argentina      5.901    47
```

finding the rank of countries in happiness

TOP10 RICH COUNTRIES

```
In [48]: top10 = (
df.groupby("country_name")["log_gdp_per_capita"]
.mean()
.sort_values(ascending=False)
.head(10)
)
print(top10)
```

```
country_name
Luxembourg      11.607091
Qatar           11.510667
Singapore       11.329000
Switzerland     11.095400
United Arab Emirates  11.066727
Ireland         11.064857
Norway          11.039100
Kuwait          10.977800
United States   10.956067
Austria         10.886846
Name: log_gdp_per_capita, dtype: float64
```

```
In [49]: gdp_mean = df.groupby("country_name")["log_gdp_per_capita"].mean()
rank = gdp_mean.rank(ascending=False)
india_rank = rank["India"]
print("India's Rank:", int(india_rank))
```

India's Rank: 122

```
In [50]: india_dt= df[df["country_name"]=="India"]
```

```
In [51]: india_dt
```

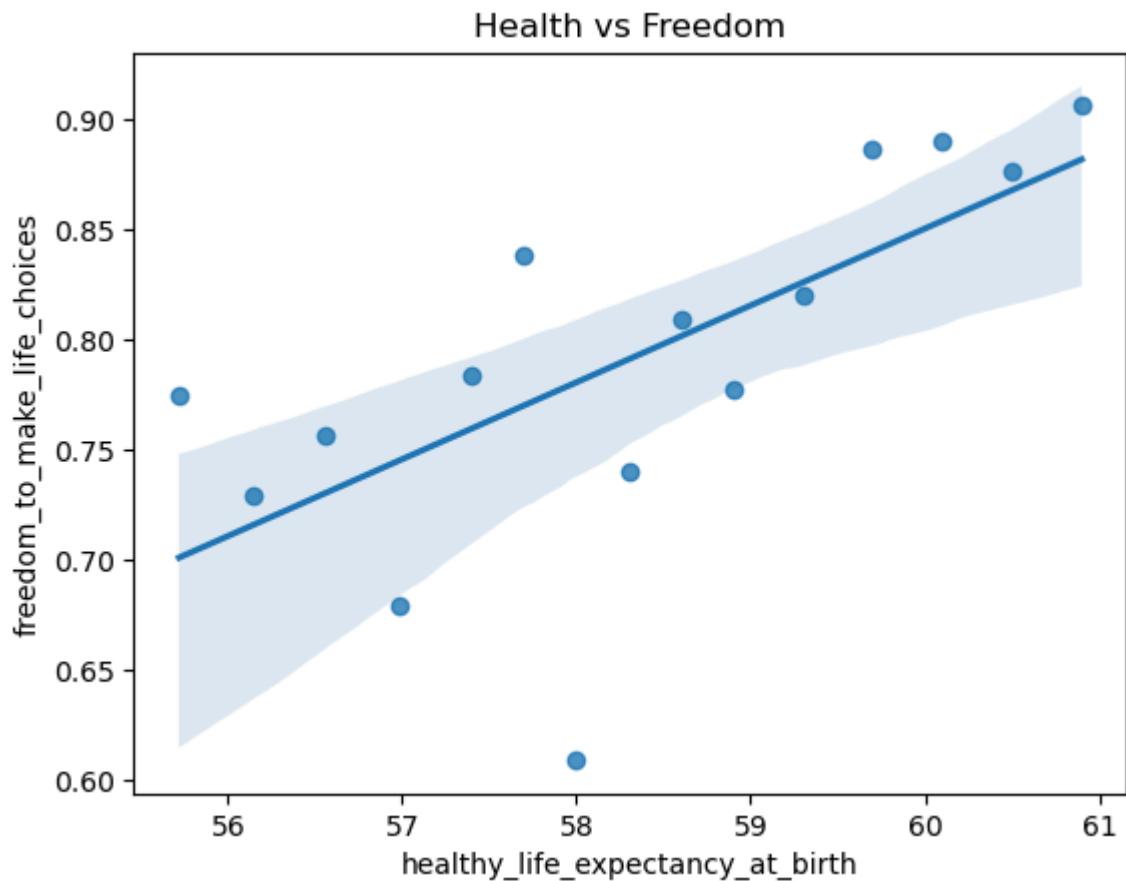
```
Out[51]:
```

	country_name	year	life_ladder	log_gdp_per_capita	social_support	healthy_life_e
726	India	2006	5.348	8.145	0.707	
727	India	2007	5.027	8.204	0.569	
728	India	2008	5.146	8.220	0.684	
729	India	2009	4.522	8.281	0.653	
730	India	2010	4.989	8.349	0.605	
731	India	2011	4.635	8.387	0.553	
732	India	2012	4.720	8.428	0.511	
733	India	2013	4.428	8.478	0.553	
734	India	2014	4.424	8.538	0.621	
735	India	2015	4.342	8.604	0.610	
736	India	2016	4.179	8.673	0.614	
737	India	2017	4.046	8.730	0.607	
738	India	2018	3.818	8.779	0.638	
739	India	2019	3.249	8.818	0.561	
740	India	2020	4.225	8.703	0.617	

```
In [52]: india_dt.mean(numeric_only=True)
```

```
Out[52]: year                2013.000000
life_ladder                4.473200
log_gdp_per_capita         8.489133
social_support             0.606867
healthy_life_expectancy_at_birth  58.320000
freedom_to_make_life_choices    0.791467
generosity                 0.017807
perceptions_of_corruption      0.828467
positive_affect             0.686667
negative_affect             0.307533
dtype: float64
```

```
In [53]: sns.regplot(data=india_dt, x="healthy_life_expectancy_at_birth", y="freedom_to_m
plt.title("Health vs Freedom")
plt.show()
```

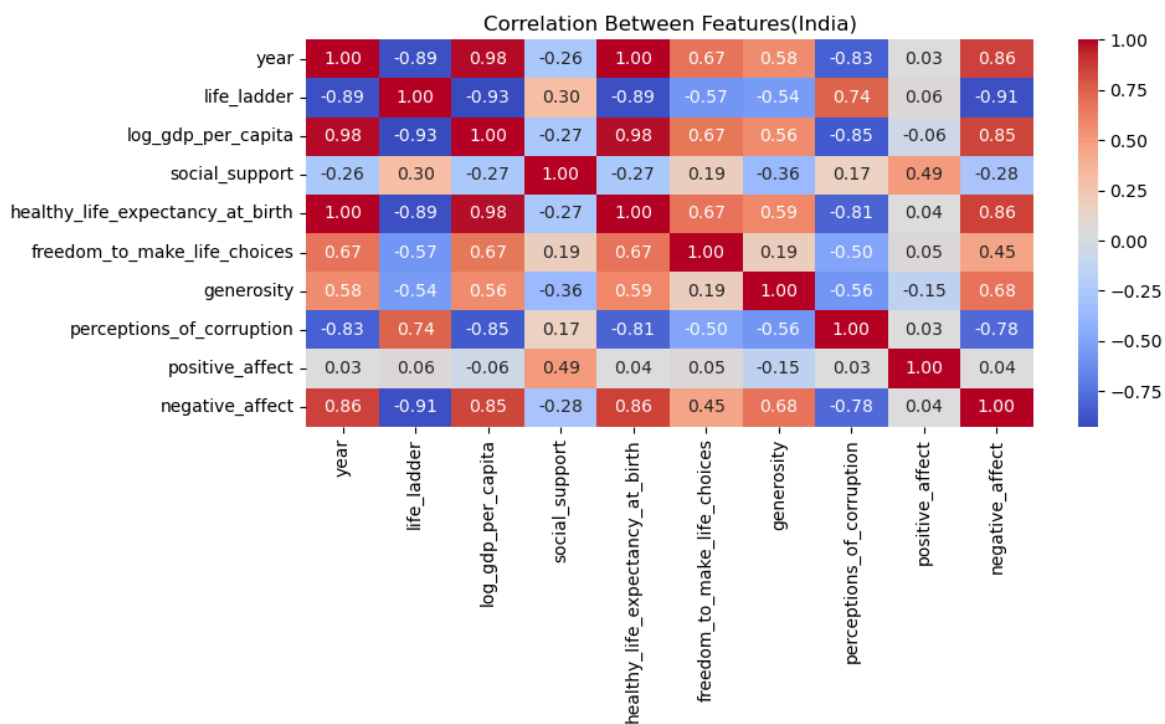


In india Health vs Freedom directly propotional

```
In [54]: plt.figure(figsize=(10,6))

sns.heatmap(india_dt.corr(numeric_only=True), annot=True, fmt=".2f", cmap="coolw

plt.title("Correlation Between Features(India)")
plt.tight_layout()
plt.show()
```



Insight

- ◆ Strong Positive Relationships (+): log_gdp_per_capita ↔ healthy_life_expectancy_at_birth : ~0.98, year ↔ log_gdp_per_capita : ~0.98, year ↔ healthy_life_expectancy_at_birth : ~1.00, life_ladder ↔ perceptions_of_corruption : ~0.74
- ◆ Strong Negative Relationships (-): life_ladder ↔ negative_affect : ~ -0.91, life_ladder ↔ log_gdp_per_capita : ~ -0.93, life_ladder ↔ year : ~ -0.89, perceptions_of_corruption ↔ log_gdp_per_capita : ~ -0.85, perceptions_of_corruption ↔ year : ~ -0.83
- ◆ Moderate Relationships: life_ladder ↔ social_support : ~0.30, life_ladder ↔ freedom_to_make_life_choices : ~ -0.57, generosity ↔ negative_affect : ~0.68, freedom_to_make_life_choices ↔ log_gdp_per_capita : ~0.67
- ◆ Weak / No Relationships: life_ladder ↔ positive_affect : ~0.06, positive_affect ↔ most variables : weak correlation
- ◆ Weak / No Relationships: life_ladder ↔ positive_affect : ~0.06, positive_affect ↔ most variables : weak correlation

Happiness (life_ladder) in India is strongly negatively correlated with negative emotions and time, while economic factors like GDP and life expectancy are strongly positively correlated with each other

```
In [55]: df = df.drop("happiness_level", axis=1)
```

```
In [56]: df
```

```
Out[56]:
```

	country_name	year	life_ladder	log_gdp_per_capita	social_support	healthy_life_e
0	Afghanistan	2008	3.724	7.370	0.451	
1	Afghanistan	2009	4.402	7.540	0.552	
2	Afghanistan	2010	4.758	7.647	0.539	
3	Afghanistan	2011	3.832	7.620	0.521	
4	Afghanistan	2012	3.783	7.705	0.521	
...	...	...	...	...	...	...
1920	Zimbabwe	2016	3.735	7.984	0.768	
1921	Zimbabwe	2017	3.638	8.016	0.754	
1922	Zimbabwe	2018	3.616	8.049	0.775	
1923	Zimbabwe	2019	2.694	7.950	0.759	
1924	Zimbabwe	2020	3.160	7.829	0.717	

1925 rows × 11 columns



Data analysis over continents

```
In [57]: pip install pycountry pycountry-convert
```

```
Requirement already satisfied: pycountry in c:\users\intern\anaconda3\lib\site-packages (26.2.16)
Requirement already satisfied: pycountry-convert in c:\users\intern\anaconda3\lib\site-packages (0.7.2)
Requirement already satisfied: pprintpp>=0.3.0 in c:\users\intern\anaconda3\lib\site-packages (from pycountry-convert) (0.4.0)
Requirement already satisfied: pytest>=3.4.0 in c:\users\intern\anaconda3\lib\site-packages (from pycountry-convert) (8.4.2)
Requirement already satisfied: pytest-mock>=1.6.3 in c:\users\intern\anaconda3\lib\site-packages (from pycountry-convert) (3.15.1)
Requirement already satisfied: pytest-cov>=2.5.1 in c:\users\intern\anaconda3\lib\site-packages (from pycountry-convert) (7.1.0)
Requirement already satisfied: repoze.lru>=0.7 in c:\users\intern\anaconda3\lib\site-packages (from pycountry-convert) (0.7)
Requirement already satisfied: wheel>=0.30.0 in c:\users\intern\anaconda3\lib\site-packages (from pycountry-convert) (0.45.1)
Requirement already satisfied: colorama>=0.4 in c:\users\intern\anaconda3\lib\site-packages (from pytest>=3.4.0->pycountry-convert) (0.4.6)
Requirement already satisfied: iniconfig>=1 in c:\users\intern\anaconda3\lib\site-packages (from pytest>=3.4.0->pycountry-convert) (2.1.0)
Requirement already satisfied: packaging>=20 in c:\users\intern\anaconda3\lib\site-packages (from pytest>=3.4.0->pycountry-convert) (25.0)
Requirement already satisfied: pluggy<2,>=1.5 in c:\users\intern\anaconda3\lib\site-packages (from pytest>=3.4.0->pycountry-convert) (1.5.0)
Requirement already satisfied: pygments>=2.7.2 in c:\users\intern\anaconda3\lib\site-packages (from pytest>=3.4.0->pycountry-convert) (2.19.2)
Requirement already satisfied: coverage>=7.10.6 in c:\users\intern\anaconda3\lib\site-packages (from coverage[toml]>=7.10.6->pytest-cov>=2.5.1->pycountry-convert) (7.13.5)
Note: you may need to restart the kernel to use updated packages.
```

```
In [58]: import pycountry
import pycountry_convert as pc
```

```
In [59]: def get_continent(country_name):
    try:
        country = pycountry.countries.lookup(country_name)
        country_code = country.alpha_2
        continent_code = pc.country_alpha2_to_continent_code(country_code)
        continent_map = {
            "AF": "Africa",
            "AS": "Asia",
            "EU": "Europe",
            "NA": "North America",
            "SA": "South America",
            "OC": "Oceania"
        }
        return continent_map.get(continent_code)
    except:
        return "Unknown"
df["continent"] = df["country_name"].apply(get_continent)
```

```
In [60]: df["continent"]
```



```
Out[60]: 0      Asia
1      Asia
2      Asia
3      Asia
4      Asia
...
1920   Africa
1921   Africa
1922   Africa
1923   Africa
1924   Africa
Name: continent, Length: 1925, dtype: object
```

```
In [61]: df
```

```
Out[61]:
```

	country_name	year	life_ladder	log_gdp_per_capita	social_support	healthy_life_e
0	Afghanistan	2008	3.724	7.370	0.451	
1	Afghanistan	2009	4.402	7.540	0.552	
2	Afghanistan	2010	4.758	7.647	0.539	
3	Afghanistan	2011	3.832	7.620	0.521	
4	Afghanistan	2012	3.783	7.705	0.521	
...	...	...	...	...	...	...
1920	Zimbabwe	2016	3.735	7.984	0.768	
1921	Zimbabwe	2017	3.638	8.016	0.754	
1922	Zimbabwe	2018	3.616	8.049	0.775	
1923	Zimbabwe	2019	2.694	7.950	0.759	
1924	Zimbabwe	2020	3.160	7.829	0.717	

1925 rows × 12 columns



```
In [62]: df["continent"].value_counts()
```

```
Out[62]: continent
Asia      512
Europe    505
Africa    440
North America  170
South America  149
Unknown    121
Oceania     28
Name: count, dtype: int64
```

```
In [63]: continent_avg = df.groupby("continent")["life_ladder"].mean().sort_values(ascending=True)
print(continent_avg)
```

```
continent
Oceania      7.296179
Europe       6.184624
North America 6.167147
South America 6.055221
Asia         5.277594
Unknown      5.213950
Africa       4.352920
Name: life_ladder, dtype: float64
```

```
In [64]: df.groupby("continent")[[
        "life_ladder",
        "log_gdp_per_capita",
        "social_support",
        "healthy_life_expectancy_at_birth"
        ]].mean()
```

```
Out[64]:
```

	life_ladder	log_gdp_per_capita	social_support	healthy_life_expectancy_at_birt
Africa	4.352920	8.117064	0.714275	54.00136
Asia	5.277594	9.333158	0.787189	64.09269
Europe	6.184624	10.348536	0.893014	69.23459
North America	6.167147	9.464838	0.847948	65.67894
Oceania	7.296179	10.669286	0.950143	72.56071
South America	6.055221	9.517479	0.868591	66.33919
Unknown	5.213950	9.323828	0.795074	60.61149

continent

Africa	4.352920	8.117064	0.714275	54.00136
Asia	5.277594	9.333158	0.787189	64.09269
Europe	6.184624	10.348536	0.893014	69.23459
North America	6.167147	9.464838	0.847948	65.67894
Oceania	7.296179	10.669286	0.950143	72.56071
South America	6.055221	9.517479	0.868591	66.33919
Unknown	5.213950	9.323828	0.795074	60.61149

◀  ▶

```
In [65]: df[df["continent"] == "Unknown"]["country_name"].unique()
```

```
Out[65]: array(['Congo (Brazzaville)', 'Congo (Kinshasa)',
        'Hong Kong S.A.R. of China', 'Ivory Coast', 'Kosovo',
        'North Cyprus', 'Palestinian Territories', 'Russia',
        'Somaliland region', 'Swaziland', 'Taiwan Province of China',
        'Turkey'], dtype=object)
```

```
In [66]: df["country_name"] = df["country_name"].replace({
        "United States": "United States of America",
        "South Korea": "Korea, Republic of",
        "North Korea": "Korea, Democratic People's Republic of",
        "Russia": "Russian Federation",
        "Vietnam": "Viet Nam",
        "Iran": "Iran, Islamic Republic of",
        "Syria": "Syrian Arab Republic",
        "Tanzania": "Tanzania, United Republic of",
        "Venezuela": "Venezuela, Bolivarian Republic of",
        "Bolivia": "Bolivia, Plurinational State of"
    })
```

```
In [67]: df["continent"] = df["country_name"].apply(get_continent)
```

```
In [68]: df["continent"].value_counts()
```

```
Out[68]: continent
Europe          520
Asia             512
Africa           440
North America   170
South America   149
Unknown         106
Oceania          28
Name: count, dtype: int64
```

```
In [69]: df["continent"].value_counts()
```

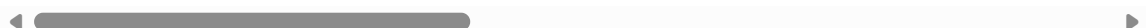
```
Out[69]: continent
Europe          520
Asia             512
Africa           440
North America   170
South America   149
Unknown         106
Oceania          28
Name: count, dtype: int64
```

```
In [70]: df
```

```
Out[70]:
```

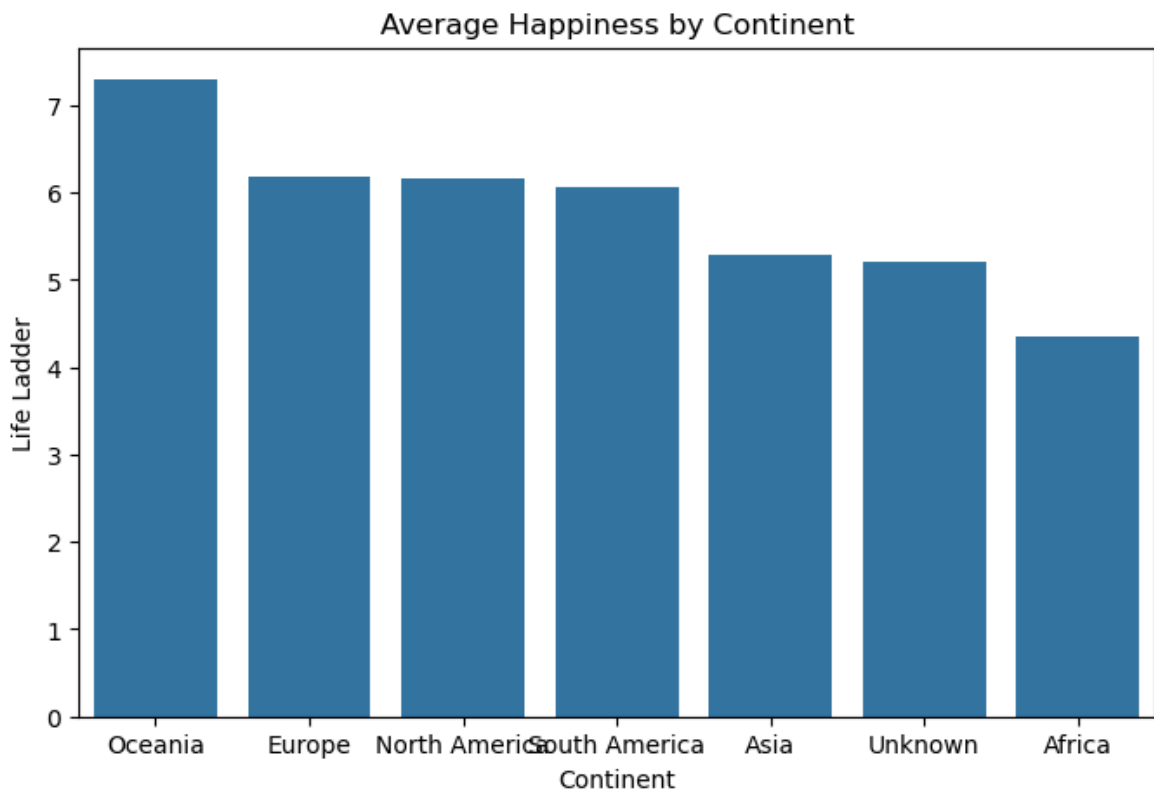
	country_name	year	life_ladder	log_gdp_per_capita	social_support	healthy_life_e
0	Afghanistan	2008	3.724	7.370	0.451	
1	Afghanistan	2009	4.402	7.540	0.552	
2	Afghanistan	2010	4.758	7.647	0.539	
3	Afghanistan	2011	3.832	7.620	0.521	
4	Afghanistan	2012	3.783	7.705	0.521	
...	...	...	...	...	...	...
1920	Zimbabwe	2016	3.735	7.984	0.768	
1921	Zimbabwe	2017	3.638	8.016	0.754	
1922	Zimbabwe	2018	3.616	8.049	0.775	
1923	Zimbabwe	2019	2.694	7.950	0.759	
1924	Zimbabwe	2020	3.160	7.829	0.717	

1925 rows × 7 columns



```
In [71]: plt.figure(figsize=(8,5))
sns.barplot(
    x=continent_avg.index,
    y=continent_avg.values
)
plt.title("Average Happiness by Continent")
plt.xlabel("Continent")
```

```
plt.ylabel("Life Ladder")
plt.show()
```



Oceania ranks highest in happiness, while Africa ranks lowest, showing regional inequality in well-being

Analysis Description

This analysis explores global happiness trends using a dataset containing indicators such as GDP per capita, social support, health, freedom, generosity, and corruption perceptions. The data was first cleaned by handling missing values and standardizing column names for consistency.

A new feature, continent, was created to enable region-wise analysis. The study examines relationships between key variables and happiness scores using visualizations like bar charts, heatmaps, density plots, and line graphs.

The analysis compares happiness levels across continents, identifies top and bottom performing countries, and studies trends over time. It also investigates how economic and social factors influence happiness both globally and within specific regions.

KEY QUESTIONS:

1.Which country is most/least happy?

Denmark is the most happy country, while countries like Afghanistan (and in some cases South Sudan) show the lowest happiness levels.

This indicates that countries with strong economic conditions, better health, and higher social support tend to have higher happiness scores, whereas countries facing economic challenges and instability tend to have lower happiness levels.

```
In [72]: top10 = (  
df.groupby("country_name")["life_ladder"]  
  .mean()  
  .sort_values(ascending=False)  
  .head(10)  
)  
print(top10)
```

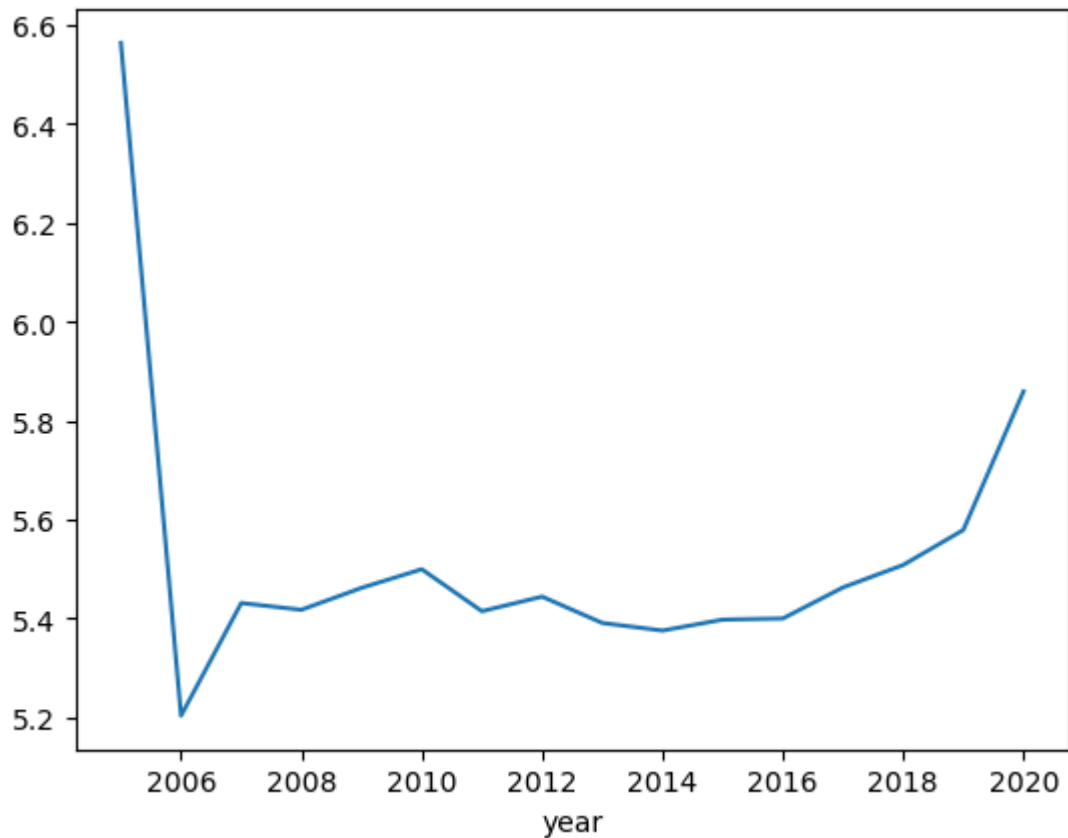
```
country_name  
Denmark      7.680400  
Finland      7.597154  
Switzerland  7.548300  
Norway       7.512400  
Netherlands  7.466286  
Iceland      7.446500  
Canada       7.376333  
Sweden       7.369467  
New Zealand  7.310286  
Australia    7.282071  
Name: life_ladder, dtype: float64
```

2.How does happiness change over time?

The results show that happiness levels remain relatively stable over time, with only small fluctuations. There is no extreme increase or decrease globally, but slight variations may occur due to economic conditions, global events, or regional differences.

```
In [73]: df.groupby("year")["life_ladder"].mean().plot()
```

```
Out[73]: <Axes: xlabel='year'>
```



3.Does GDP affect happiness?

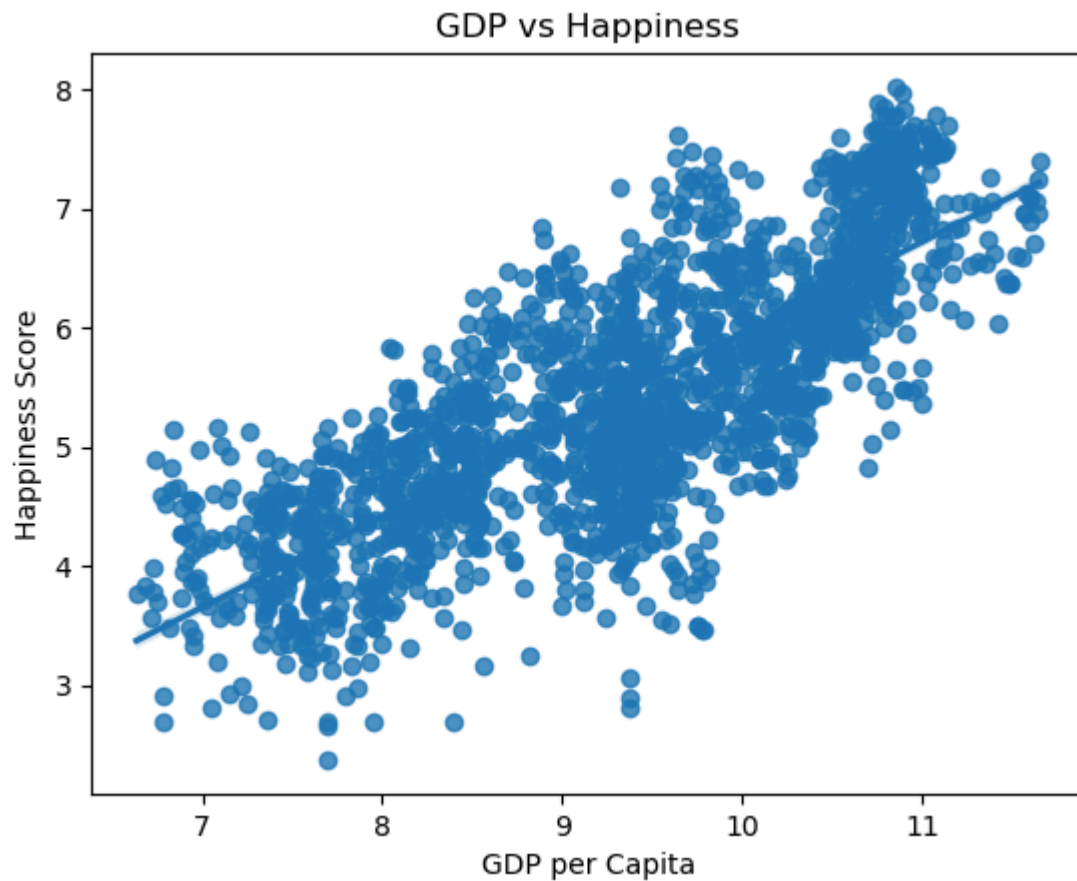
The analysis shows a strong positive relationship, meaning that countries with higher GDP per capita generally have higher happiness levels.

This suggests that better economic conditions improve living standards, access to resources, and overall well-being, which contributes to higher happiness.

```
In [74]: df["log_gdp_per_capita"].corr(df["life_ladder"])
```

```
Out[74]: np.float64(0.7840856439520998)
```

```
In [75]: sns.regplot(x='log_gdp_per_capita', y='life_ladder', data=df)
plt.xlabel("GDP per Capita")
plt.ylabel("Happiness Score")
plt.title("GDP vs Happiness")
plt.show()
```



4.Can we predict happiness using other features?

Yes, happiness can be predicted using other features such as GDP per capita, social support, health, freedom, generosity, and corruption perception

```
In [76]: sns.pairplot(  
df[[  
"life_ladder",  
"log_gdp_per_capita",  
"social_support",  
"healthy_life_expectancy_at_birth",  
"freedom_to_make_life_choices"  
]]  
)  
plt.show()
```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[76], line 10
      1 sns.pairplot(
      2 df[[
      3 "life_ladder",
      (...)      8 ]]
      9 )
--> 10 plt.show()

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\pyplot.py:614, in show(*args, **kwargs)
    570 """
    571 Display all open figures.
    572
    (...)    611 explicitly there.
    612 """
    613 _warn_if_gui_out_of_main_thread()
--> 614 return _get_backend_mod().show(*args, **kwargs)

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib_inline\backend_inline.py:90, in show(close, block)
     88 try:
     89     for figure_manager in Gcf.get_all_fig_managers():
--> 90         display(
     91             figure_manager.canvas.figure,
     92             metadata=_fetch_figure_metadata(figure_manager.canvas.figure),
     93         )
     94 finally:
     95     show._to_draw = []

File c:\Users\INTERN\anaconda3\Lib\site-packages\IPython\core\display_functions.py:278, in display(include, exclude, metadata, transient, display_id, raw, clear, *objs, **kwargs)
    276     publish_display_data(data=obj, metadata=metadata, **kwargs)
    277 else:
--> 278     format_dict, md_dict = format(obj, include=include, exclude=exclude)
    279     if not format_dict:
    280         # nothing to display (e.g. _ipython_display_ took over)
    281         continue

File c:\Users\INTERN\anaconda3\Lib\site-packages\IPython\core\formatters.py:238, in DisplayFormatter.format(self, obj, include, exclude)
    236 md = None
    237 try:
--> 238     data = formatter(obj)
    239 except:
    240     # FIXME: log the exception
    241     raise

File c:\Users\INTERN\anaconda3\Lib\site-packages\decorator.py:235, in decorate.<locals>.fun(*args, **kw)
    233 if not kwsyntax:
    234     args, kw = fix(args, kw, sig)
--> 235 return caller(func, *(extras + args), **kw)

File c:\Users\INTERN\anaconda3\Lib\site-packages\IPython\core\formatters.py:282, in catch_format_error(method, self, *args, **kwargs)
    280 """show traceback on failed format call"""

```



```

281 try:
--> 282     r = method(self, *args, **kwargs)
283 except NotImplementedError:
284     # don't warn on NotImplementedError
285     return self._check_return(None, args[0])

```

File c:\Users\INTERN\anaconda3\Lib\site-packages\IPython\core\formatters.py:402, in BaseFormatter.__call__(self, obj)

```

400     pass
401 else:
--> 402     return printer(obj)
403 # Finally look for special method names
404 method = get_real_method(obj, self.print_method)

```

File c:\Users\INTERN\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:170, in print_figure(fig, fmt, bbox_inches, base64, **kwargs)

```

167     from matplotlib.backend_bases import FigureCanvasBase
168     FigureCanvasBase(fig)
--> 170 fig.canvas.print_figure(bytes_io, **kw)
171 data = bytes_io.getvalue()
172 if fmt == 'svg':

```

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\backend_bases.py:2186, in FigureCanvasBase.print_figure(self, filename, dpi, facecolor, edgecolor, orientation, format, bbox_inches, pad_inches, bbox_extra_artists, backend, **kwargs)

```

2182 try:
2183     # _get_renderer may change the figure dpi (as vector formats
2184     # force the figure dpi to 72), so we need to set it again here.
2185     with cbook._setattr_cm(self.figure, dpi=dpi):
-> 2186         result = print_method(
2187             filename,
2188             facecolor=facecolor,
2189             edgecolor=edgecolor,
2190             orientation=orientation,
2191             bbox_inches_restore=_bbox_inches_restore,
2192             **kwargs)
2193 finally:
2194     if bbox_inches and restore_bbox:

```

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\backend_bases.py:2042, in FigureCanvasBase._switch_canvas_and_return_print_method.<locals>.<lambda>(*args, **kwargs)

```

2038     optional_kws = { # Passed by print_figure for other renderers.
2039         "dpi", "facecolor", "edgecolor", "orientation",
2040         "bbox_inches_restore"}
2041     skip = optional_kws - {*inspect.signature(meth).parameters}
-> 2042     print_method = functools.wraps(meth)(lambda *args, **kwargs: meth(
2043         *args, **{k: v for k, v in kwargs.items() if k not in skip}))
2044 else: # Let third-parties do as they see fit.
2045     print_method = meth

```

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\backends\backend_agg.py:481, in FigureCanvasAgg.print_png(self, filename_or_obj, metadata, pil_kwargs)

```

434 def print_png(self, filename_or_obj, *, metadata=None, pil_kwargs=None):
435     """
436     Write the figure to a PNG file.
437
438     (...) 479     *metadata*, including the default 'Software' key.
439     """

```

```
--> 481 self._print_pil(filename_or_obj, , pil_kwargs, metadata)
```

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\backends\backend_agg.py:429, in FigureCanvasAgg._print_pil(self, filename_or_obj, fmt, pil_kwargs, metadata)

```
424 def _print_pil(self, filename_or_obj, fmt, pil_kwargs, metadata=None):
425     """
426     Draw the canvas, then save it using `image.imsave` (to which
427     *pil_kwargs* and *metadata* are forwarded).
428     """
--> 429 FigureCanvasAgg.draw(self)
430     mpl.image.imsave(
431         filename_or_obj, self.buffer_rgba(), format=fmt, origin="upper",
432         dpi=self.figure.dpi, metadata=metadata, pil_kwargs=pil_kwargs)
```

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\backends\backend_agg.py:382, in FigureCanvasAgg.draw(self)

```
379 # Acquire a lock on the shared font cache.
380 with (self.toolbar._wait_cursor_for_draw_cm() if self.toolbar
381       else nullcontext()):
--> 382 self.figure.draw(self.renderer)
383     # A GUI class may be need to update a window using this draw, so
384     # don't forget to call the superclass.
385     super().draw()
```

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\artist.py:94, in _finalize_rasterization.<locals>.draw_wrapper(artist, renderer, *args, **kwargs)

```
92 @wraps(draw)
93 def draw_wrapper(artist, renderer, *args, **kwargs):
---> 94     result = draw(artist, renderer, *args, **kwargs)
95     if renderer._rasterizing:
96         renderer.stop_rasterizing()
```

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\artist.py:71, in allow_rasterization.<locals>.draw_wrapper(artist, renderer)

```
68 if artist.get_agg_filter() is not None:
69     renderer.start_filter()
---> 71 return draw(artist, renderer)
72 finally:
73     if artist.get_agg_filter() is not None:
```

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\figure.py:3257, in Figure.draw(self, renderer)

```
3254 # ValueError can occur when resizing a window.
3255 self.patch.draw(renderer)
-> 3257 mimage._draw_list_compositing_images(
3258     renderer, self, artists, self.suppressComposite)
3260 renderer.close_group('figure')
3261 finally:
```

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\image.py:134, in _draw_list_compositing_images(renderer, parent, artists, suppress_composite)

```
132 if not_composite or not has_images:
133     for a in artists:
--> 134         a.draw(renderer)
135 else:
136     # Composite any adjacent images together
137     image_group = []
```

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\artist.py:71, in allow

```

w_rasterization.<locals>.draw_wrapper(artist, renderer)
    68     if artist.get_agg_filter() is not None:
    69         renderer.start_filter()
---> 71     return draw(artist, renderer)
    72 finally:
    73     if artist.get_agg_filter() is not None:

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\axes\_base.py:3226, in
n_AxesBase.draw(self, renderer)
    3223 if artists_rasterized:
    3224     _draw_rasterized(self.get_figure(root=True), artists_rasterized, rend
erer)
-> 3226 mimage._draw_list_compositing_images(
    3227     renderer, self, artists, self.get_figure(root=True).suppressComposit
e)
    3229 renderer.close_group('axes')
    3230 self.stale = False

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\image.py:134, in _dra
w_list_compositing_images(renderer, parent, artists, suppress_composite)
    132 if not_composite or not has_images:
    133     for a in artists:
--> 134         a.draw(renderer)
    135 else:
    136     # Composite any adjacent images together
    137     image_group = []

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\artist.py:71, in allo
w_rasterization.<locals>.draw_wrapper(artist, renderer)
    68     if artist.get_agg_filter() is not None:
    69         renderer.start_filter()
---> 71     return draw(artist, renderer)
    72 finally:
    73     if artist.get_agg_filter() is not None:

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\axis.py:1411, in Axi
s.draw(self, renderer)
    1408     tick.draw(renderer)
    1410 # Shift label away from axes to avoid overlapping ticklabels.
-> 1411 self._update_label_position(renderer)
    1412 self.label.draw(renderer)
    1414 self._update_offset_text_position(tlb1, tlb2)

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\axis.py:2454, in XAxi
s._update_label_position(self, renderer)
    2449 x, y = self.label.get_position()
    2451 if self.label_position == 'bottom':
    2452     # Union with extents of the bottom spine if present, of the axes othe
rwise.
    2453     bbox = mtransforms.Bbox.union([
-> 2454         *bboxes, self.axes.spines.get(
ent()), self.axes).get_window_ext
ent()])
    2455     self.label.set_position(
    2456         (x, bbox.y0 - self.labelpad * self.get_figure(root=True).dpi / 7
2))
    2457 else:
    2458     # Union with extents of the top spine if present, of the axes otherwi
se.

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\spines.py:158, in Spi

```

```

ne.get_window_extent(self, renderer)
    156     return bb
    157 bboxes = [bb]
--> 158 drawn_ticks = self.axis._update_ticks()
    160 major_tick = next(iter(*drawn_ticks} & {*self.axis.majorTicks}), None)
    161 minor_tick = next(iter(*drawn_ticks} & {*self.axis.minorTicks}), None)

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\axis.py:1288, in Axis._update_ticks(self)
    1286     tick.label1.set_text(label)
    1287     tick.label2.set_text(label)
-> 1288 minor_locs = self.get_minorticklocs()
    1289 minor_labels = self.minor.formatter.format_ticks(minor_locs)
    1290 minor_ticks = self.get_minor_ticks(len(minor_locs))

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\axis.py:1532, in Axis.get_minorticklocs(self)
    1530 minor_locs = np.asarray(self.minor.locator())
    1531 if self.remove_overlapping_locs:
-> 1532     major_locs = self.major.locator()
    1533     transform = self._scale.get_transform()
    1534     tr_minor_locs = transform.transform(minor_locs)

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\ticker.py:2230, in MaxNLocator.__call__(self)
    2228 def __call__(self):
    2229     vmin, vmax = self.axis.get_view_interval()
-> 2230     return self.tick_values(vmin, vmax)

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\ticker.py:2238, in MaxNLocator.tick_values(self, vmin, vmax)
    2235     vmin = -vmax
    2236 vmin, vmax = mtransforms.nonsingular(
    2237     vmin, vmax, expander=1e-13, tiny=1e-14)
-> 2238 locs = self._raw_ticks(vmin, vmax)
    2240 prune = self._prune
    2241 if prune == 'lower':

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\ticker.py:2168, in MaxNLocator._raw_ticks(self, vmin, vmax)
    2166 if self._nbins == 'auto':
    2167     if self.axis is not None:
-> 2168         nbins = np.clip(self.axis.get_tick_space(),
    2169                         max(1, self._min_n_ticks - 1), 9)
    2170 else:
    2171     nbins = 9

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\axis.py:2581, in XAxis.get_tick_space(self)
    2580 def get_tick_space(self):
-> 2581     ends = mtransforms.Bbox.unit().transformed(
    2582         self.axes.transAxes - self.get_figure(root=False).dpi_scale_trans
s)
    2583     length = ends.width * 72
    2584     # There is a heuristic here that the aspect ratio of tick text
    2585     # is no more than 3:1

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\transforms.py:466, in BboxBase.transformed(self, transform)
    462 """

```

```

463 Construct a `Bbox` by statically transforming this one by *transform*.
464 """
465 pts = self.get_points()
--> 466 ll, ul, lr = transform.transform(np.array(
467     [pts[0], [pts[0, 0], pts[1, 1]], [pts[1, 0], pts[0, 1]]))
468 return Bbox([ll, [lr[0], ul[1]]])

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\transforms.py:1496, i
n Transform.transform(self, values)
1493 values = values.reshape((-1, self.input_dims))
1495 # Transform the values
-> 1496 res = self.transform_affine(self.transform_non_affine(values))
1498 # Convert the result back to the shape of the input values.
1499 if ndim == 0:

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\transforms.py:2411, i
n CompositeGenericTransform.transform_affine(self, values)
2409 def transform_affine(self, values):
2410     # docstring inherited
-> 2411     return self.get_affine().transform(values)

File c:\Users\INTERN\anaconda3\Lib\site-packages\matplotlib\transforms.py:2437, i
n CompositeGenericTransform.get_affine(self)
2435     return self._b.get_affine()
2436 else:
-> 2437     return Affine2D(np.dot(self._b.get_affine().get_matrix(),
2438                             self._a.get_affine().get_matrix()))

KeyboardInterrupt:

```

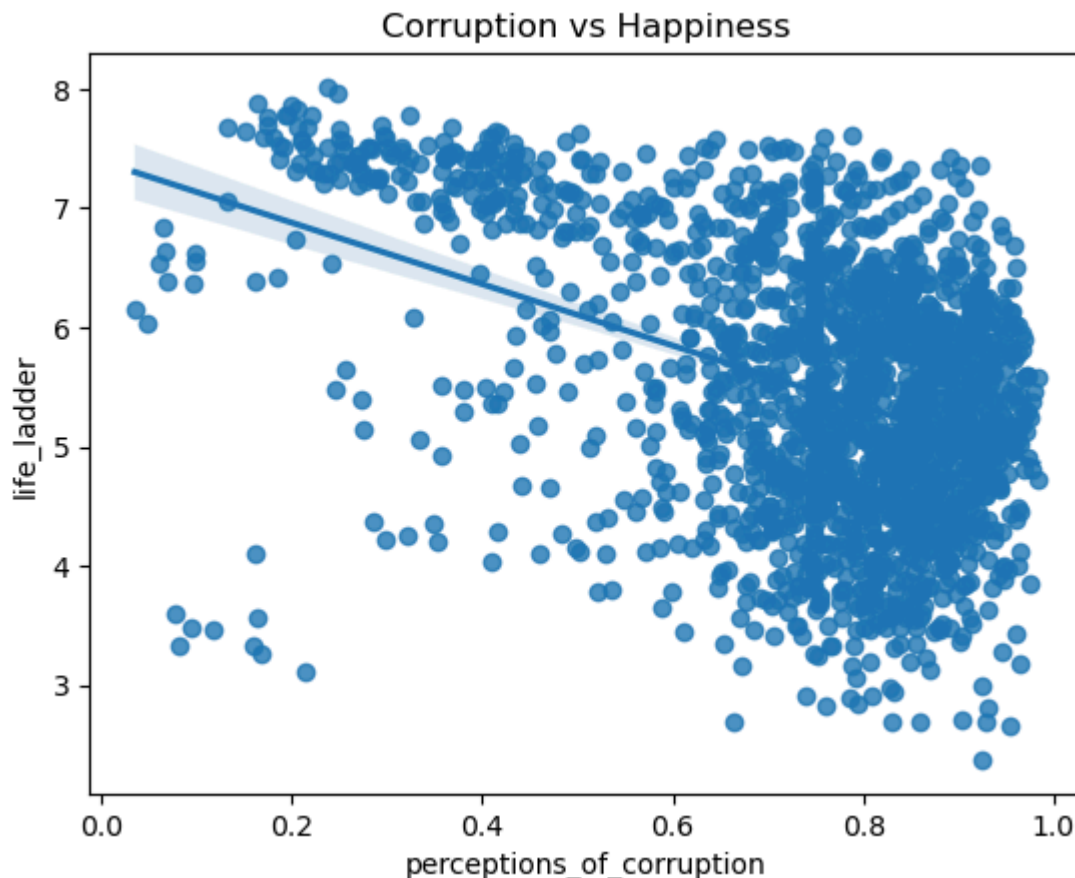
5.How does corruption affect happiness?

The analysis shows a negative relationship between corruption and happiness. This means that as the perception of corruption increases, the happiness level tends to decrease

```
In [ ]: df["perceptions_of_corruption"].corr(df["life_ladder"])
```

```
Out[ ]: np.float64(-0.4201607727593406)
```

```
In [ ]: sns.regplot(data=df, x="perceptions_of_corruption", y="life_ladder")
plt.title("Corruption vs Happiness")
plt.show()
```



6.How does happiness vary across continents?

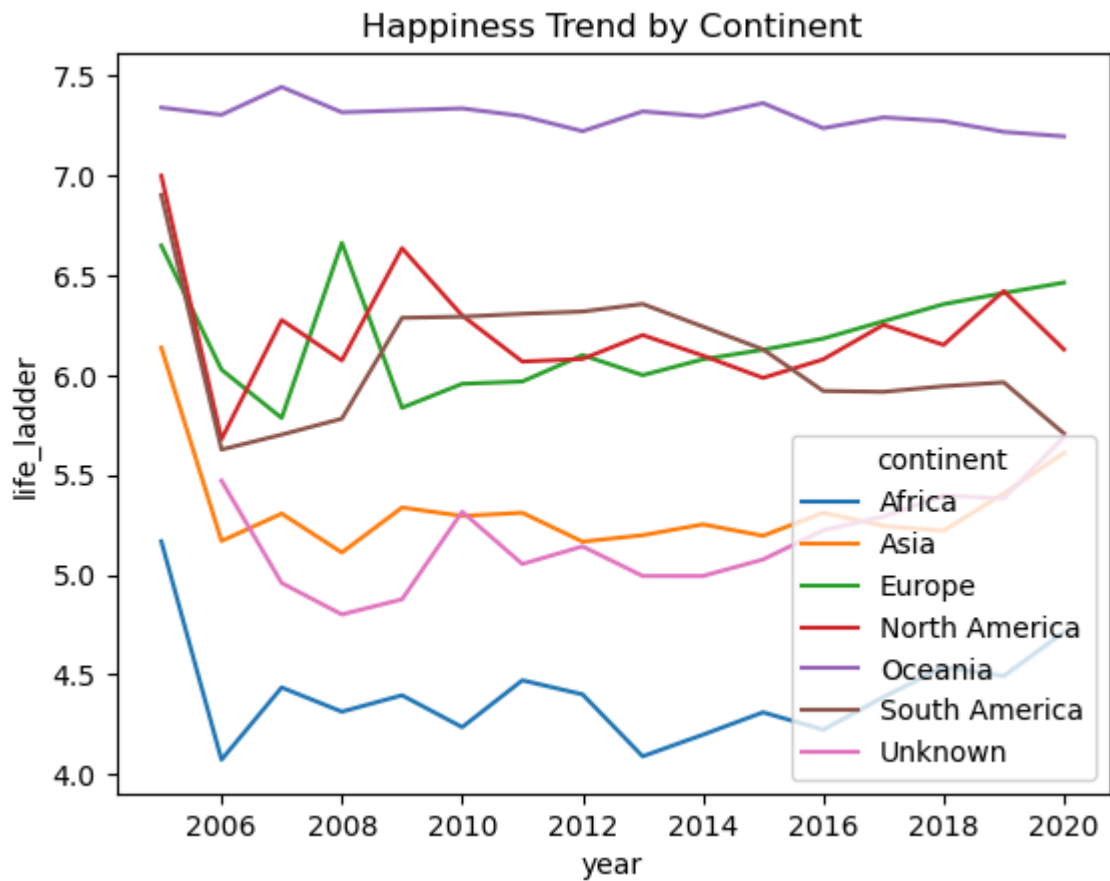
Happiness levels vary significantly across different continents. By grouping the data by continent and calculating the average life_ladder score, clear regional differences can be observed.

👉 Oceania shows the highest happiness levels, indicating strong economic conditions, health, and social support. 👉 Europe and North America also have high happiness scores, reflecting developed economies and better quality of life. 👉 South America has moderate happiness levels, slightly lower but relatively stable. 👉 Asia shows mixed results due to variation between developed and developing countries. 👉 Africa has the lowest happiness levels, likely due to lower GDP, health challenges, and limited resources.

```
In [77]: continent_trend = df.groupby(["year", "continent"])["life_ladder"].mean().reset_index()

sns.lineplot(
    data=continent_trend,
    x="year",
    y="life_ladder",
    hue="continent"
)

plt.title("Happiness Trend by Continent")
plt.show()
```



In [78]: df

Out[78]:

	country_name	year	life_ladder	log_gdp_per_capita	social_support	healthy_life_e
0	Afghanistan	2008	3.724	7.370	0.451	
1	Afghanistan	2009	4.402	7.540	0.552	
2	Afghanistan	2010	4.758	7.647	0.539	
3	Afghanistan	2011	3.832	7.620	0.521	
4	Afghanistan	2012	3.783	7.705	0.521	
...	...	...	...	...	...	...
1920	Zimbabwe	2016	3.735	7.984	0.768	
1921	Zimbabwe	2017	3.638	8.016	0.754	
1922	Zimbabwe	2018	3.616	8.049	0.775	
1923	Zimbabwe	2019	2.694	7.950	0.759	
1924	Zimbabwe	2020	3.160	7.829	0.717	

1925 rows × 12 columns

